สัปดาห์ที่ 2 · 04_MultiAccount_Skill

การสร้างสกิล สำหรับระบบหลายบัญชี

yolo · cyolo · agent-team

ให้ AI ใช้ **yolo1–5** และ **cyolo1–5** ได้ถูกวิธี · ทำงานเป็นทีม · รู้สถิติทุกบัญชี · สลับหัวหน้าได้เมื่อโควตาหมด



พรอมต์เดียวกันใน Claude Code app · เพิ่มแค่สกิล

810.2 → **170.5**

วินาที · ก่อนมีสกิล

วินาที · หลังมีสกิล

เร็วขึ้น 4.8 เท่า · ไม่แตะไฟล์ credential · ตัวเลขโทเค็นมาจาก ledger จริง

สารบัญ

บทนี้มี 5 ส่วน

1

ปัญหา

มีคำสั่งครบ แต่ AI ยังใช้ไม่เป็น

2

แนวคิด

ทำไมสกิลแก้ได้ · สกิล 3 ตัว

3

ทีมทำงานร่วมกัน

รูปแบบ · สถิติ · โควตา · สลับหัวหน้า

4

เก็บและติดตั้ง

volume + โฟลเดอร์สำรอง · คัดลอกไปวาง

5

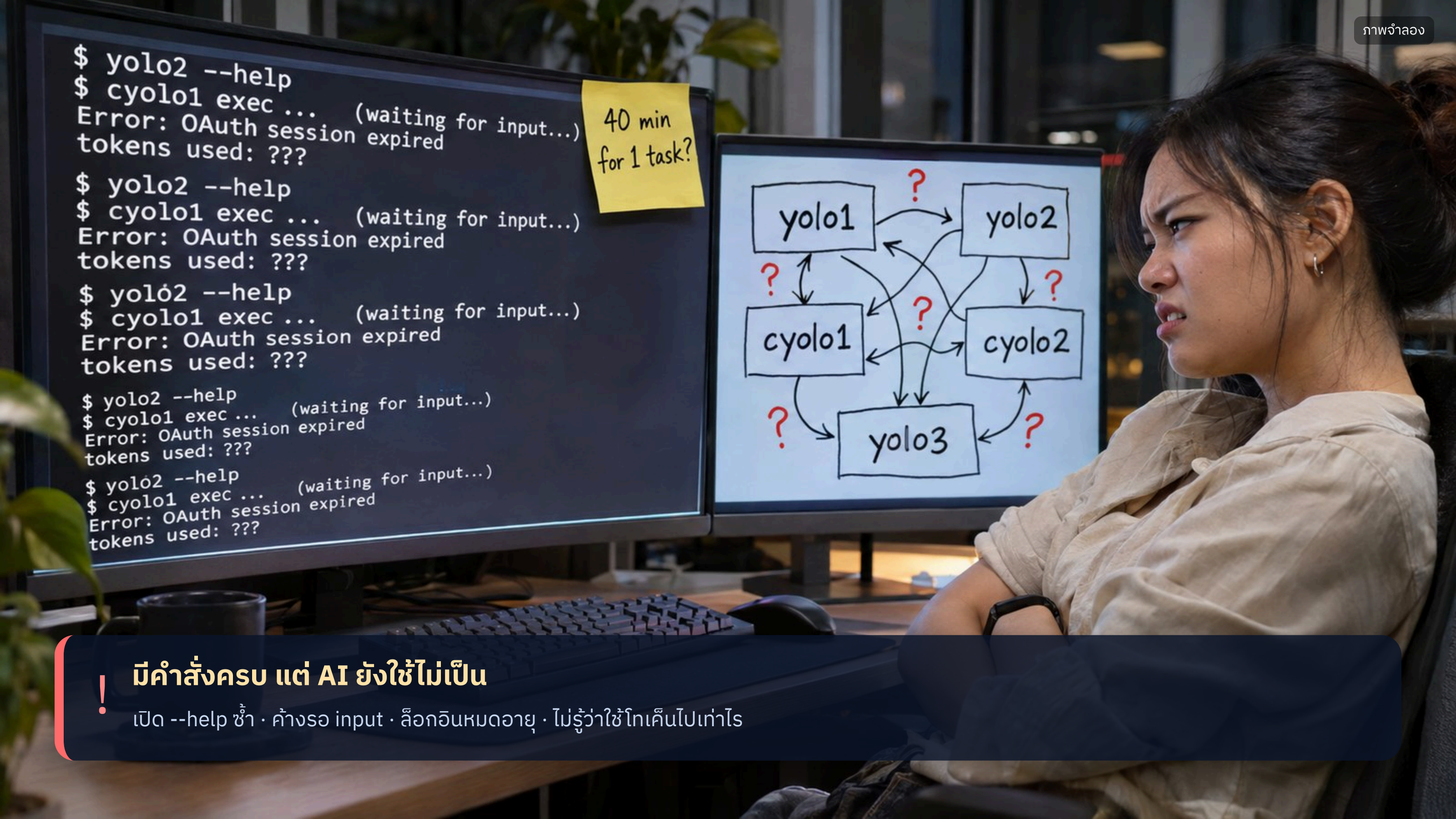
ลงมือ 4 ขั้น

สร้าง → ติดตั้ง → ทดสอบ → สลับหัวหน้า

ส่วนที่ 1 จาก 5

ปัญหา

container มี **yolo1–5** และ **cyolo1–5** ครบ แต่ AI ยังใช้ไม่เป็น · ทดลองจริงก่อนมีสกิล

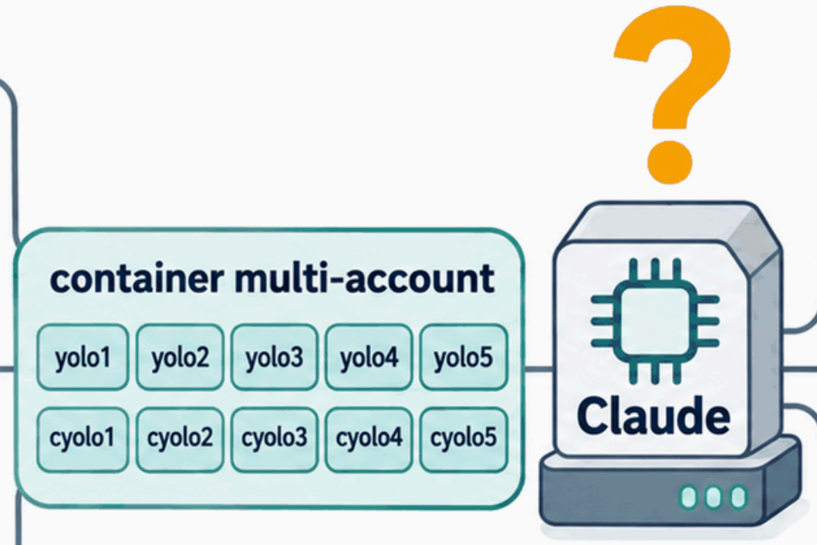

ภาพจำลอง
$ yolo2 --help
$ cyolo1 exec ... (waiting for input...)
Error: OAuth session expired
tokens used: ???
$ yolo2 --help
$ cyolo1 exec ... (waiting for input...)
Error: OAuth session expired
tokens used: ???
$ yolo2 --help
$ cyolo1 exec ... (waiting for input...)
Error: OAuth session expired
tokens used: ???
$ yolo2 --help
$ cyolo1 exec ... (waiting for input...)
Error: OAuth session expired
tokens used: ???
$ yolo2 --help
$ cyolo1 exec ... (waiting for input...)
Error: OAuth session expired
tokens used: ???
40 min for 1 task?
yolo1
yolo2
cyolo1
cyolo2
yolo3
มีคำสั่งครบ แต่ AI ยังใช้ไม่เป็น
เปิด --help ซ้ำ · ค้างรอ input · ล็อกอินหมดอายุ · ไม่รู้ว่าใช้โทเค็นไปเท่าไร

มีคำสั่งครบ แต่ AI ยังใช้ไม่เป็น

เรียกผิดวิธี
ค้างรอ input · เปิดโหมดโต้ตอบ

สถานะล็อกอินไม่ตรงจริง
loggedIn = true แต่หมดอายุ

ประสานงานไม่เป็น
รอทีละตัว · ส่งข้อมูลยาวใน prompt



ไม่มีสถิติ
tokens · เวลา · โควตา ต้องเดา

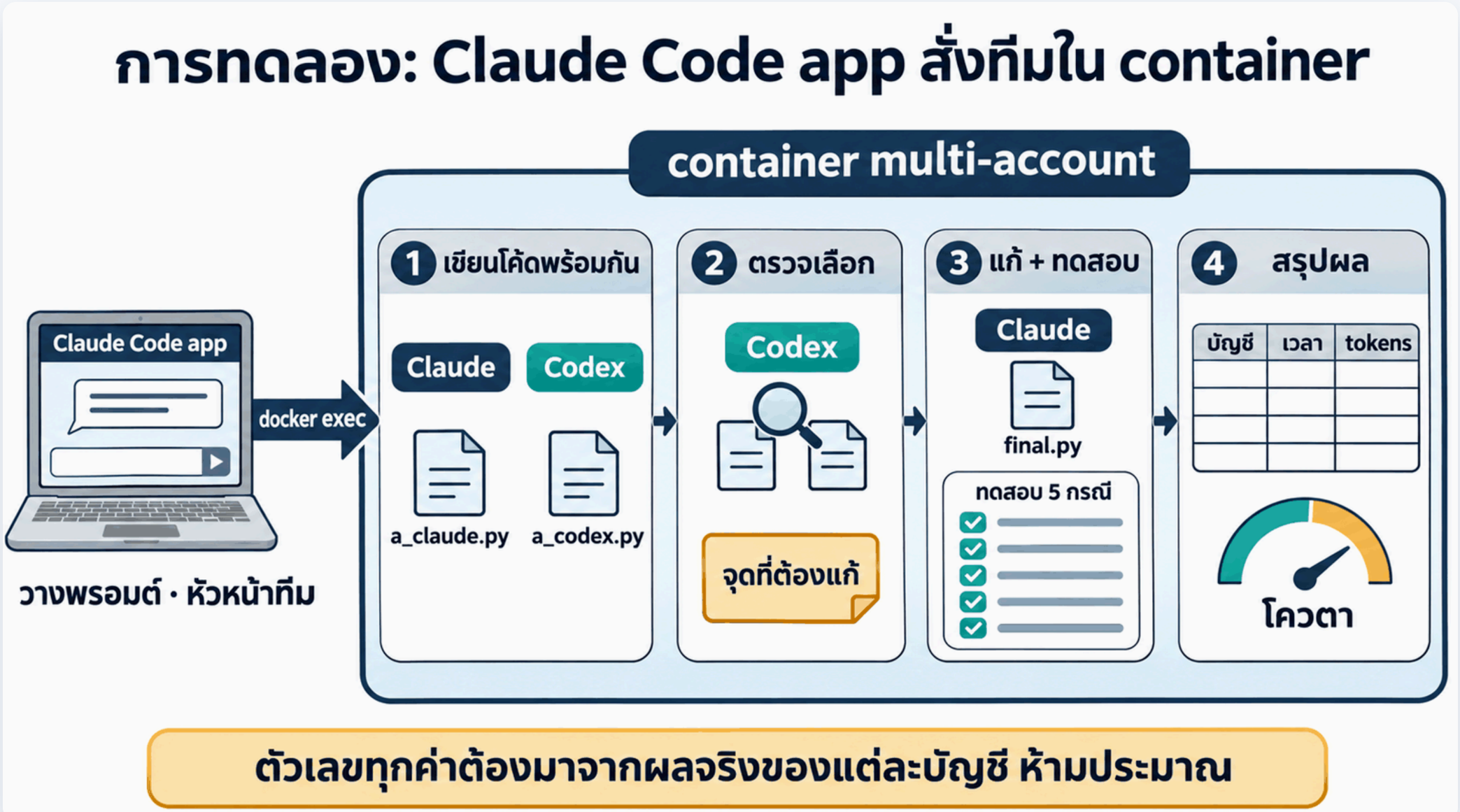
ลองผิดลองถูกทุกครั้ง
ช้า · เปลืองโทเค็น

หัวหน้าทีมโควตาหมด
แผนงานหาย · งานค้างกลางทาง

1 · ปัญหา

สิ่งที่เกิดขึ้นจริง

เพนพอยต์	สิ่งที่เกิดขึ้นจริง
เรียกผิดวิธี	`codex exec` ไม่ปิด stdin จะค้าง · ไม่ใส่ `-p` เปิดโหมดโต้ตอบ · exit code 0 ≠ สำเร็จ
สถานะล็อกอินไม่ตรงจริง	`auth status` ตอบ loggedIn แต่เรียกจริงได้ `OAuth session expired`
ประสานงานไม่เป็น	สั่งทีละบัญชี · ยัดข้อมูลยาวลง prompt · ไม่มีที่แชร์ความรู้
ไม่มีสถิติ	ไม่รู้โทเค็น เวลา และโควตาของแต่ละบัญชี
ลองผิดลองถูกทุกครั้ง	เปิด --help ซ้ำ เขียนสคริปต์ช่วยใหม่ทุกครั้ง
หัวหน้าทีมโควตาหมด	แผนงานอยู่ในบริบทแชตของหัวหน้า งานค้างหายไปด้วย

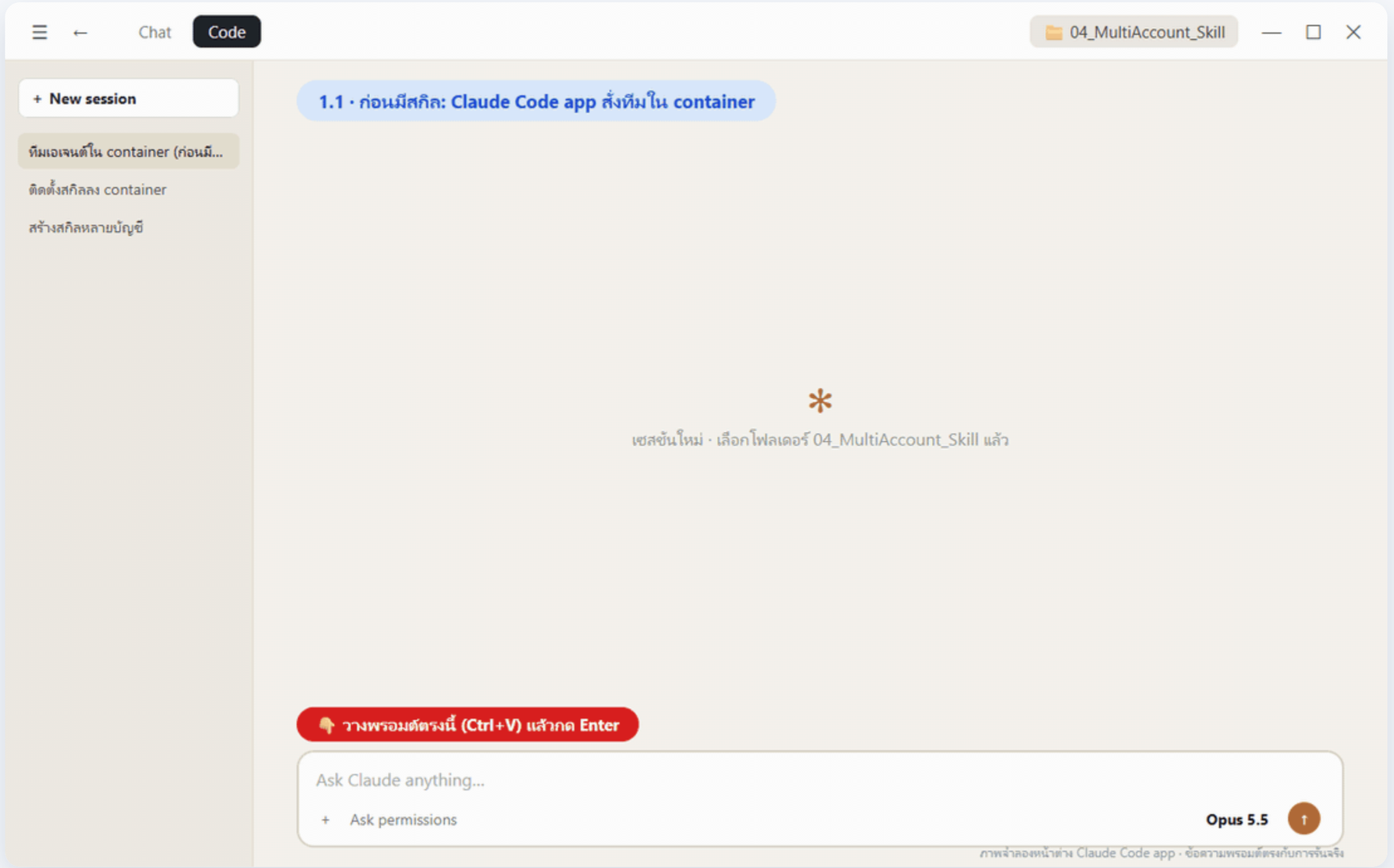

การทดลอง: Claude Code app สั่งทีมใน container
container multi-account
Claude Code app
docker exec
วางพรอมต์ · หัวหน้าทีม
1 เขียนโค้ดพร้อมกัน
Claude
Codex
a_claude.py
a_codex.py
2 ตรวจเลือก
Codex
จุดที่ต้องแก้
3 แก้ + ทดสอบ
Claude
final.py
ทดสอบ 5 กรณี
4 สรุปผล
บัญชี
เวลา
tokens
โควตา
ตัวเลขทุกค่าต้องมาจากผลจริงของแต่ละบัญชี ห้ามประมาณ

1.1 · พรอมต์ (ใช้ชุดเดียวกันทั้งก่อนและหลังมีสกิล)

```
ใช้ทีมเอเจนต์ใน container multi-account ทำงานต่อไปนี้ ทำงานใน container ผ่าน docker exec เท่านั้น (บัญชี yolo1-5 และ cyolo1-5 ใช้เฉพาะบัญชีที่พร้อม)
ก่อนเริ่ม ดูว่าใน container มีสกิลที่ช่วยงานนี้ได้หรือไม่ (/root/ai-accounts/.Claude/shared/skills) ถ้ามีให้อ่านและใช้
โฟลเดอร์งานใน container: /workspace/team-demo
1. ให้บัญชี Claude 1 บัญชี และบัญชี Codex 1 บัญชี เขียนฟังก์ชัน Python parse_duration(text) ที่แปลง "1h30m", "45s", "2h" เป็นจำนวนวินาที คนละแบบพร้อมกัน บันทึกเป็น a_claude.py และ a_codex.py
2. ส่งทั้งสองไฟล์ให้บัญชี Codex อีกบัญชีหนึ่งเลือกแบบที่ดีกว่าและบอกจุดที่ต้องแก้
3. ส่งคำแนะนำนั้นให้บัญชี Claude แก้เป็น final.py แล้วรันทดสอบ 5 กรณี
4. สรุปตาราง บัญชี | งาน | เวลา (วินาที) | input tokens | output tokens | สำเร็จ/ล้ม ตัวเลขต้องมาจากผลจริงของแต่ละบัญชี ห้ามประมาณ
5. บอกโควตาที่เหลือของแต่ละบัญชีที่ใช้
```

บรรทัด "ก่อนเริ่ม ดูว่าใน container มีสกิล..." ทำให้เทียบได้ยุติธรรม และเป็นวิธีที่ Claude Code app รู้จักสกิลใน container

1.1 · 💻 Claude Code app · ยังไม่มีสกิล



1.1 · ผลจริงก่อนมีสกิล

810.2 วินาที · 30 คำสั่ง (ล้ม 9)

✻ Claude · 1.1 · ก่อนมีสกิล: Claude Code app สั่งทีมใน container · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

เสร็จครบทั้ง 5 ขั้น ทุกขั้นรันใน container `multi-account` ผ่าน `docker exec` ไฟล์งานอยู่ที่ `/workspace/team-demo`

สกิล: ใน `/root/ai-accounts/.Claude/shared/skills` มี devtools-lab, k8s-lab, reverse-engineer-anything, shared-check และ vastai แต่ไม่มีตัวไหนช่วยงานนี้ได้ จึงเขียนสคริปต์ `run_agent.py` เองไว้รันแต่ละบัญชีแบบไม่โต้ตอบ สคริปต์จับเวลาและอ่านจำนวน token จาก JSON ที่ CLI ส่งออกมา ผลดิบเก็บไว้ที่ `logs/*.json`

ผลแต่ละขั้น

1. **เขียนโค้ดพร้อมกัน:** yolo1 เขียน `a_claude.py` และ cyolo1 เขียน `a_codex.py` ทั้งสองใช้ regex `fullmatch` แบบเดียวกัน
2. **รีวิว:** cyolo2 เลือก **a_codex.py** เพราะใช้ `[0-9]` ซึ่งรับเฉพาะเลข ASCII ส่วน `\d` รับเลขยูนิโค้ดอื่นด้วย cyolo2 ตอบว่าไม่มีจุดที่ต้องแก้ตามสเปก
3. **final.py:** yolo2 ใช้ตรรกะเดิมของ a_codex.py ที่ปรับแค่ให้อ่านง่ายขึ้น (compile regex ไว้ครั้งเดียว และตั้งชื่อตัวคูณหน่วย) แล้วรัน `test_final.py` ได้ **5/5 passed** ผมรันซ้ำเองอีกรอบแล้วผลตรงกัน
4. กรณีทดสอบ: `1h30m` →5400, `45s` →45, `2h` →7200, `1h2m3s` →3723, `""` →ValueError

ตารางสรุป (ค่าจริงจาก log)

บัญชี	งาน	เวลา (วินาที)	input tokens	output tokens	สำเร็จ/ล้ม
yolo1 (Claude)	เขียน a_claude.py	29.08 (CLI วัดได้ 15.38)	92,978	584	สำเร็จ
cyolo1 (Codex)	เขียน a_codex.py	16.69	46,615	482	สำเร็จ
cyolo2 (Codex)	รีวิวและเลือกแบบ	12.16	36,309	226	สำเร็จ
yolo2 (Claude)	final.py และทดสอบ 5 กรณี	12.96 (CLI วัดได้ 10.34)	85,946	1,052	สำเร็จ (5/5)

วิธีอ่านตัวเลข

- **เวลา:** วัดตั้งแต่เริ่มจนจบโปรเซส ตัวเลขในวงเล็บคือ `duration_ms` ที่ Claude CLI รายงานเอง yolo1 ใช้เวลานานกว่าเพราะเริ่มพร้อม cyolo1 จึงรวมเวลาเปิดตัวด้วย
- **Input ของ Claude** = input + cache_creation + cache_read
- yolo1: 6 + 20,002 + 72,970
- yolo2: 6 + 17,674 + 68,266
- **Input ของ Codex** นับ cached รวมไว้แล้ว: cyolo1 cached 43,136 และ cyolo2 cached 30,080
- **ค่าใช้จ่ายที่ CLI รายงาน:** yolo1 $0.186, yolo2 $0.176

โควตาที่เหลือ (ดึงจริงหลังจบงาน เวลาเป็นเวลาไทย)

บัญชี	5 ชั่วโมง	7 วัน	หมายเหตุ
yolo1	ไม่มีลิมิตนี้	ใช้ 22% → **เหลือ 78%** (รีเซ็ต 12 พ.ย. 06:00)	—
yolo2	ใช้ 1% → **เหลือ 99%** (รีเซ็ต 8 พ.ย. 11:10)	ใช้ 2% → **เหลือ 98%** (รีเซ็ต 9 พ.ย. 13:00)	—
cyolo1	—	ใช้ 1% → **เหลือ 99%** (รีเซ็ต 14 พ.ย. ~12:21)	—
cyolo2	—	ใช้ 1% → **เหลือ 99%** (เหมือน cyolo1)	—

ไฟล์ทั้งหมดอยู่ใน `/workspace/team-demo`: `a_claude.py`, `a_codex.py`, `review.txt`, `final.py`, `test_final.py` และ `logs/`

สิ่งที่สังเกตได้	รายละเอียด
สำรวจระบบเอง	ไล่เปิด container อื่น · Git Bash แปลง path จนล้ม
ยุ่งกับ credential	ตรวจ .credentials.json · **เปิด auth.json**
อ่านโค้ดเบื้องหลัง	อ่าน wrapper เพื่อเดาวิธีเรียก
เขียนเครื่องมือเอง	run_agent.py แล้ว docker cp เข้า container

ส่วนที่ 2 จาก 5

แนวคิดและสถาปัตยกรรม

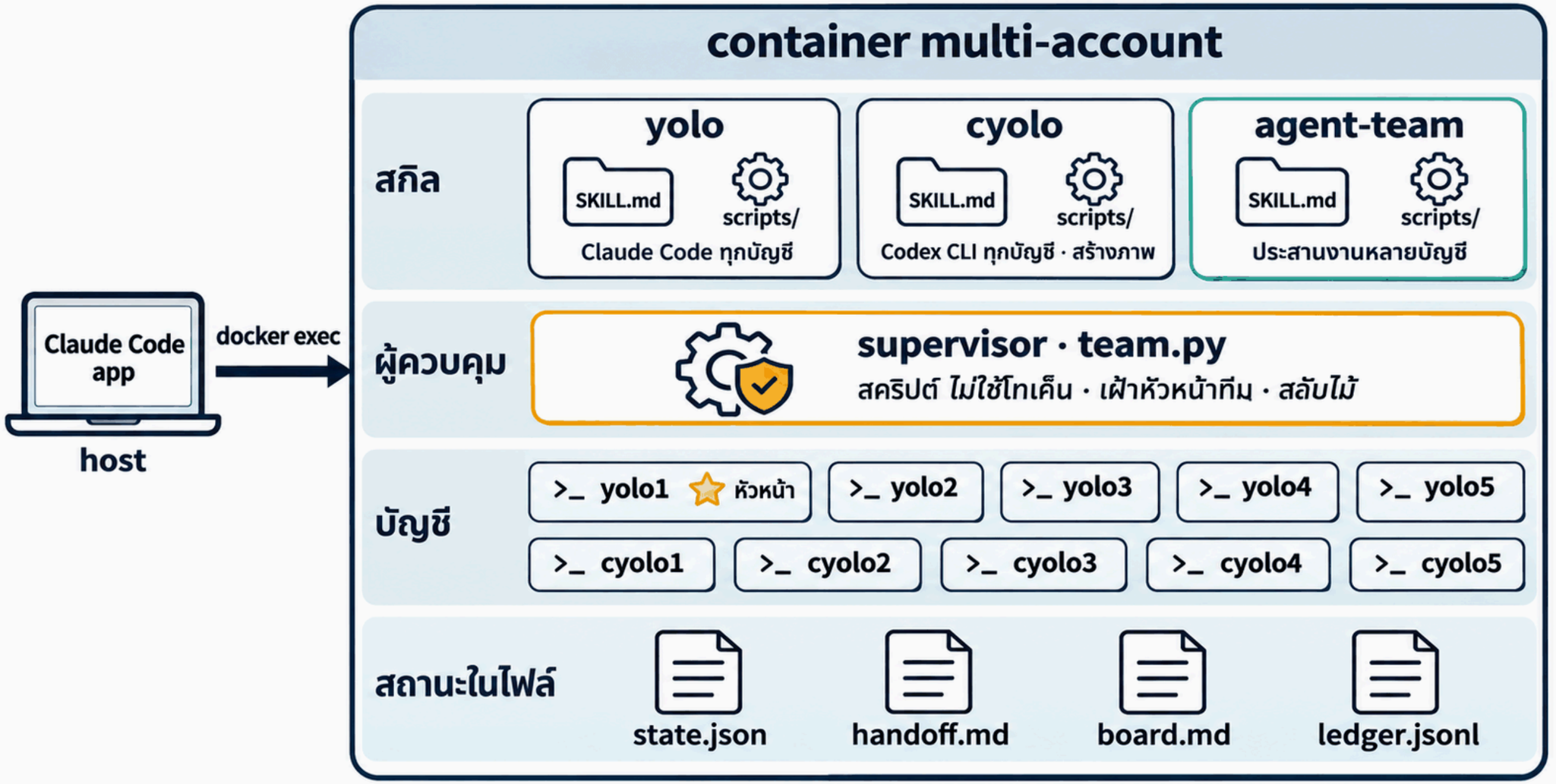
ทำไมสกิลแก้ปัญหานี้ได้ · สกิล 3 ตัว · Claude Code app เข้าใจระบบใน container อย่างไร



งานซ้ำให้สคริปต์ · งานคิดให้ AI
เรียกบัญชีให้ถูก flag ตรวจความพร้อม จับเวลา รวมสถิติ = สคริปต์ในสกิล

เปิดทีละชั้น
AI เห็นแค่ชื่อและคำอธิบายตลอดเวลา เปิด SKILL.md เมื่อจะใช้ และสคริปต์เมื่อถึงขั้นนั้น

สถาปัตยกรรมของสกิล

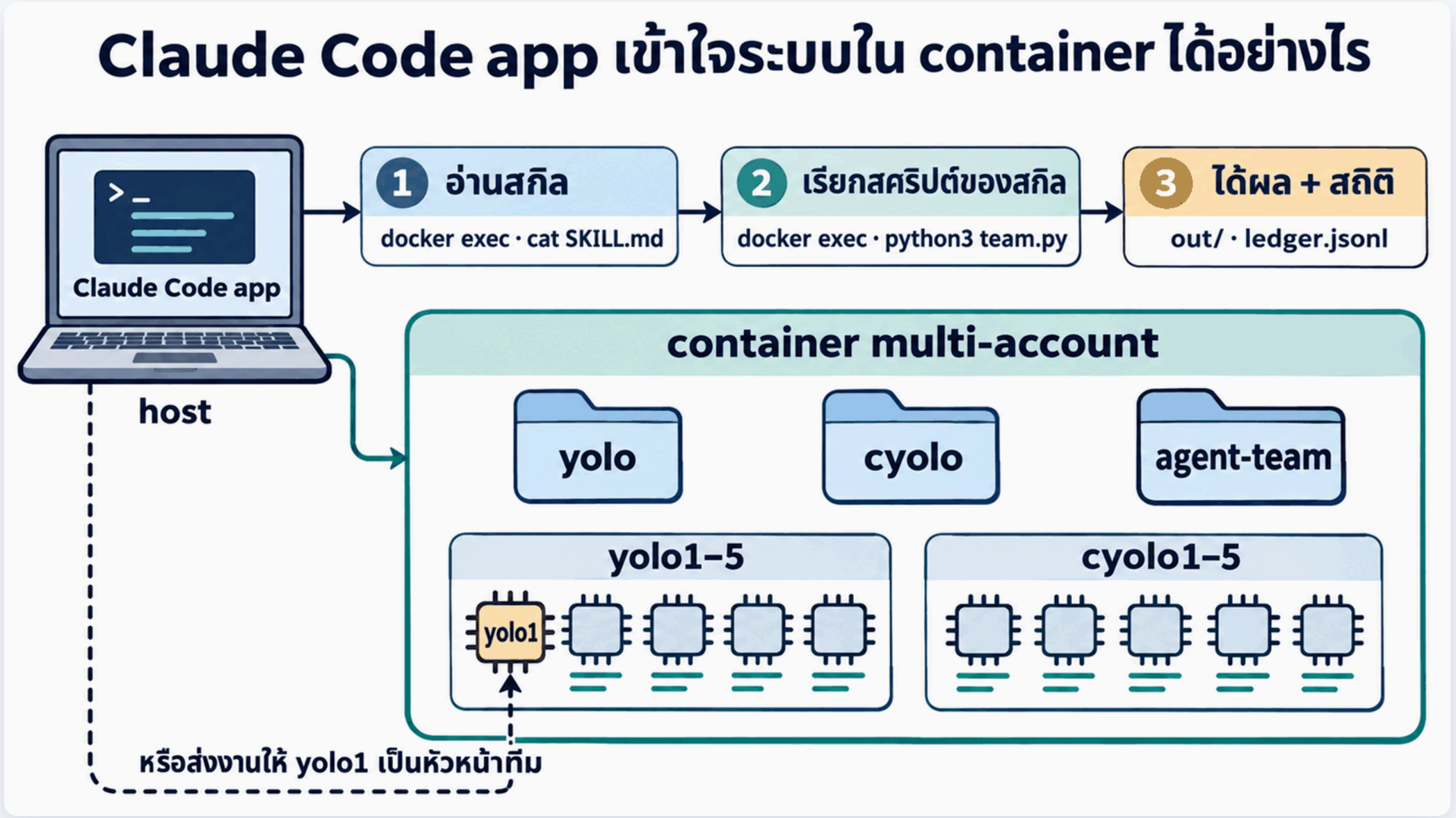


2 · ออกแบบสกิล 3 ตัว

แต่ละสกิลรู้เรื่องอะไร

สกิล	รู้เรื่องอะไร
`yolo`	Claude Code ทุกบัญชี · รุ่น/effort ตามความยาก · ตรวจความพร้อมด้วยการเรียกจริง · โควตา
`cyolo`	Codex ทุกบัญชี · รุ่นจาก models_cache · **สร้างภาพ** · โควตาจาก rollout
`agent-team`	ทำงานร่วมกันทั้งใน CLI เดียวกันและข้าม CLI · บริหารโทเค็น · สลับหัวหน้า · สถิติทั้งทีม

```
skills/
├── yolo/        SKILL.md · scripts/ · references/
├── cyolo/       SKILL.md · scripts/ · references/
└── agent-team/  SKILL.md · scripts/team.py · references/
```

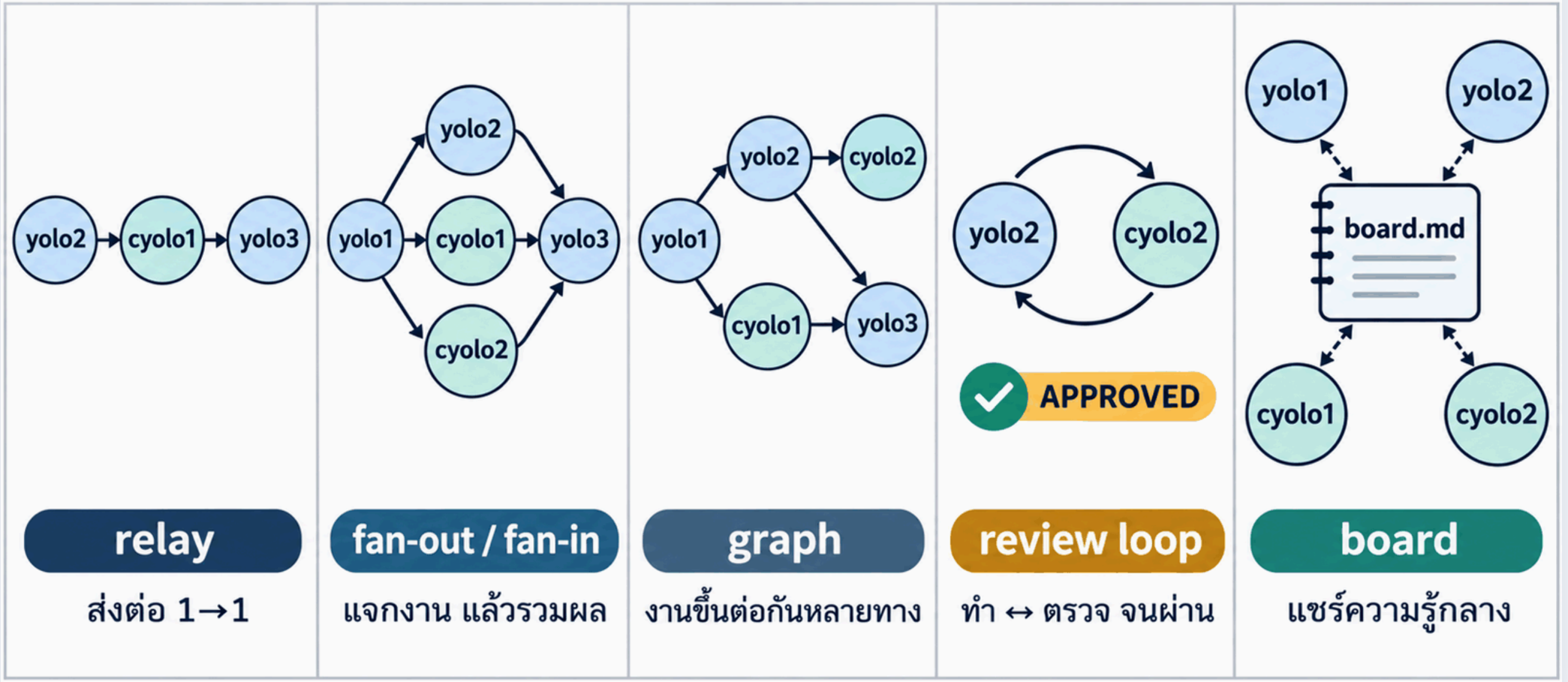

Claude Code app เข้าใจระบบใน container ได้อย่างไร
Claude Code app
host
1 อ่านสกิล
docker exec · cat SKILL.md
2 เรียกสคริปต์ของสกิล
docker exec · python3 team.py
3 ได้ผล + สถิติ
out/ · ledger.jsonl
container multi-account
yolo
cyolo
agent-team
yolo1–5
yolo1
cyolo1–5
หรือส่งงานให้ yolo1 เป็นหัวหน้าทีม

ส่วนที่ 3 จาก 5

ทีมเอเจนต์ทำงานร่วมกัน

รูปแบบการทำงานร่วมกัน · สถิติทุกบัญชี · บริหารโทเค็นของลูกทีม · สลับหัวหน้าทีม

รูปแบบการทำงานร่วมกันของหลายบัญชี



3.1 · รูปแบบการทำงานร่วมกัน

relay · ส่งต่อ 1→1

ผลของงานก่อนหน้าเป็น input ของงานถัดไป

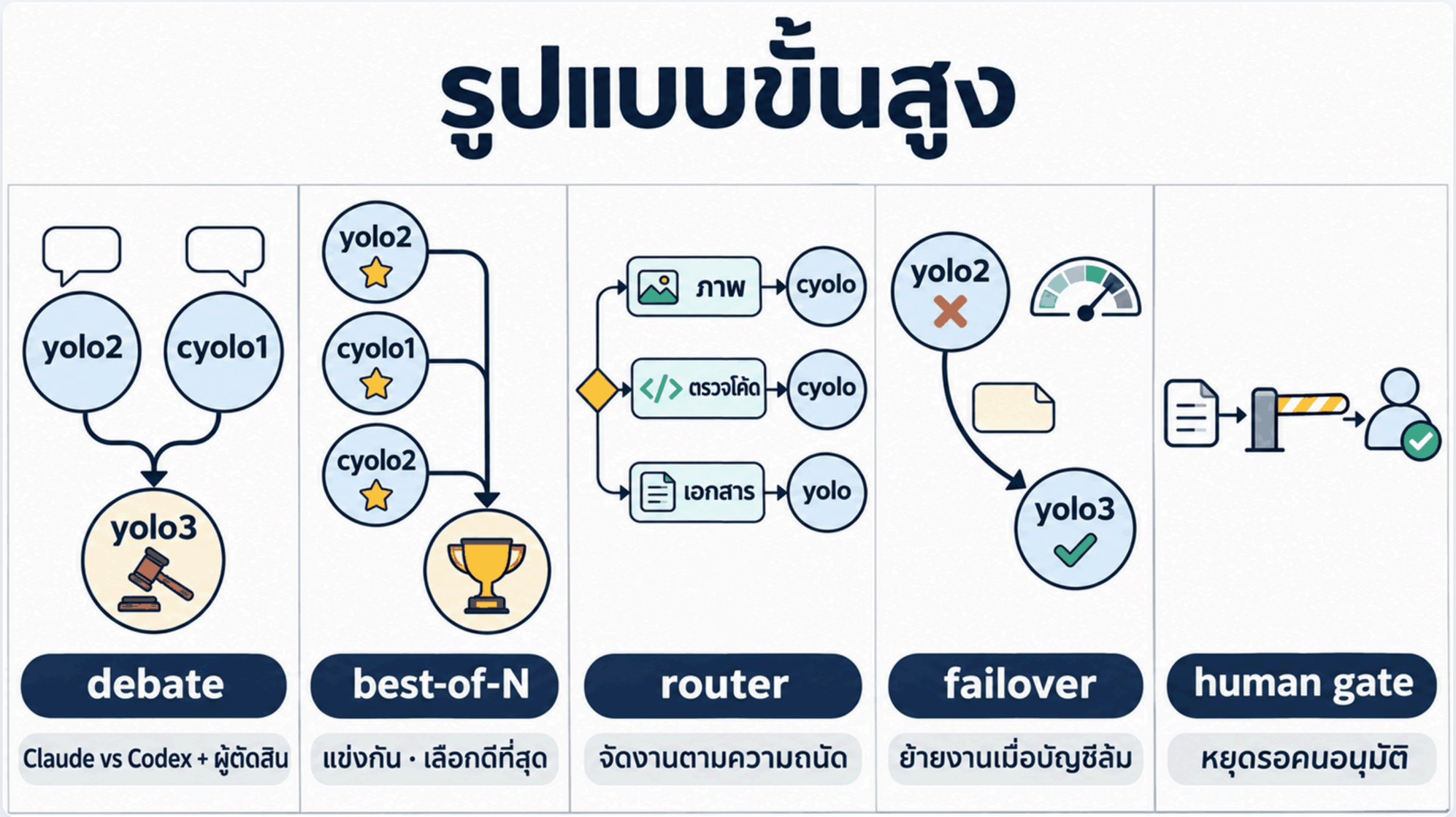
3.1 · รูปแบบการทำงานร่วมกัน

ใช้เมื่อไร และส่งข้อมูลอย่างไร

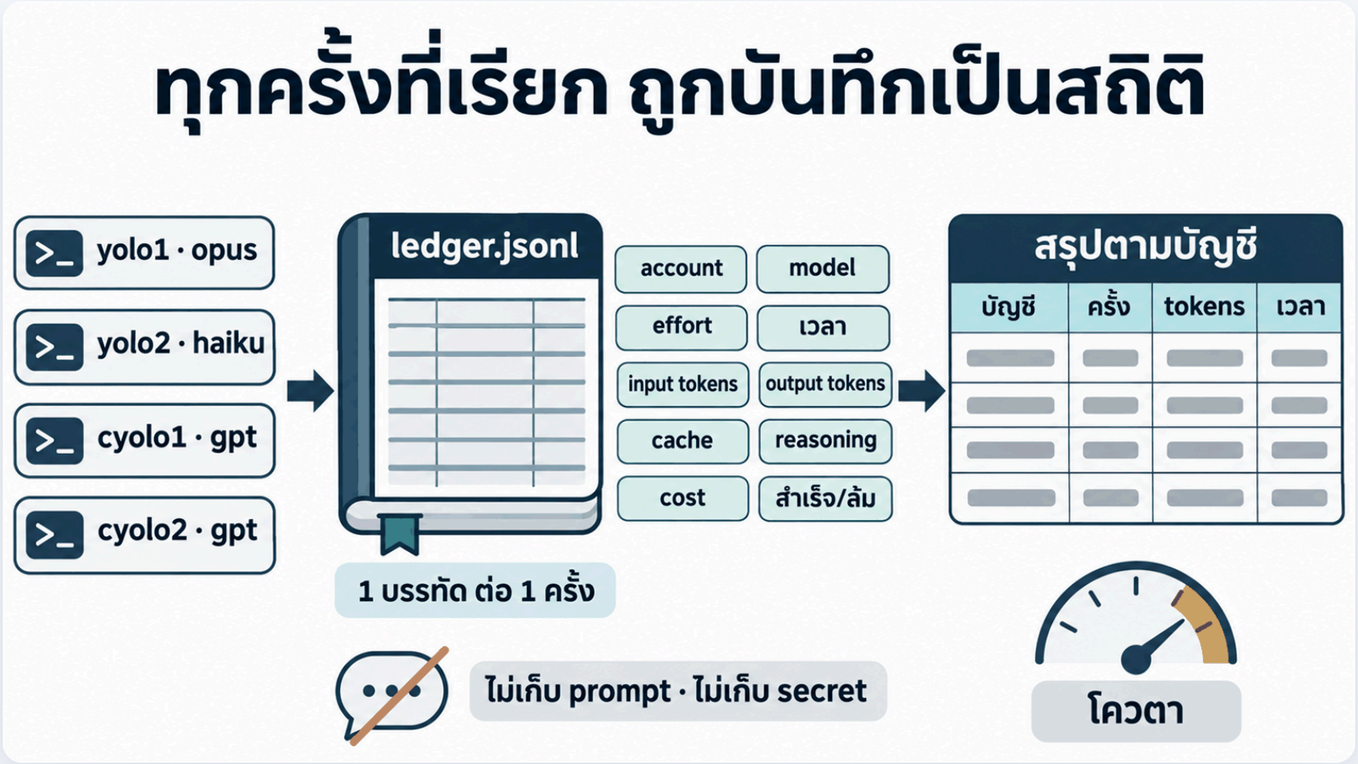
รูปแบบ	ใช้เมื่อ	การส่งข้อมูล
relay	งานเป็นลำดับ	`{{node}}` ค่าสั้นแทรกลง prompt
fan-out / fan-in	งานย่อยไม่ขึ้นต่อกัน	งานรวมผลอ่าน `{{file:node}}`
graph	งานขึ้นต่อกันหลายทาง	`depends_on` ตัวจัดคิวรันงานที่พร้อมให้เอง
review loop	ต้องการคุณภาพ	วนจน `APPROVED` หรือครบ max_rounds
board	แชร์ข้อค้นพบให้ agent ที่มาทีหลัง	อ่าน board.md ก่อนเริ่ม · เพิ่มต่อท้าย

> ข้อมูลยาวส่งเป็น **path ของไฟล์** ไม่ยัดลง prompt

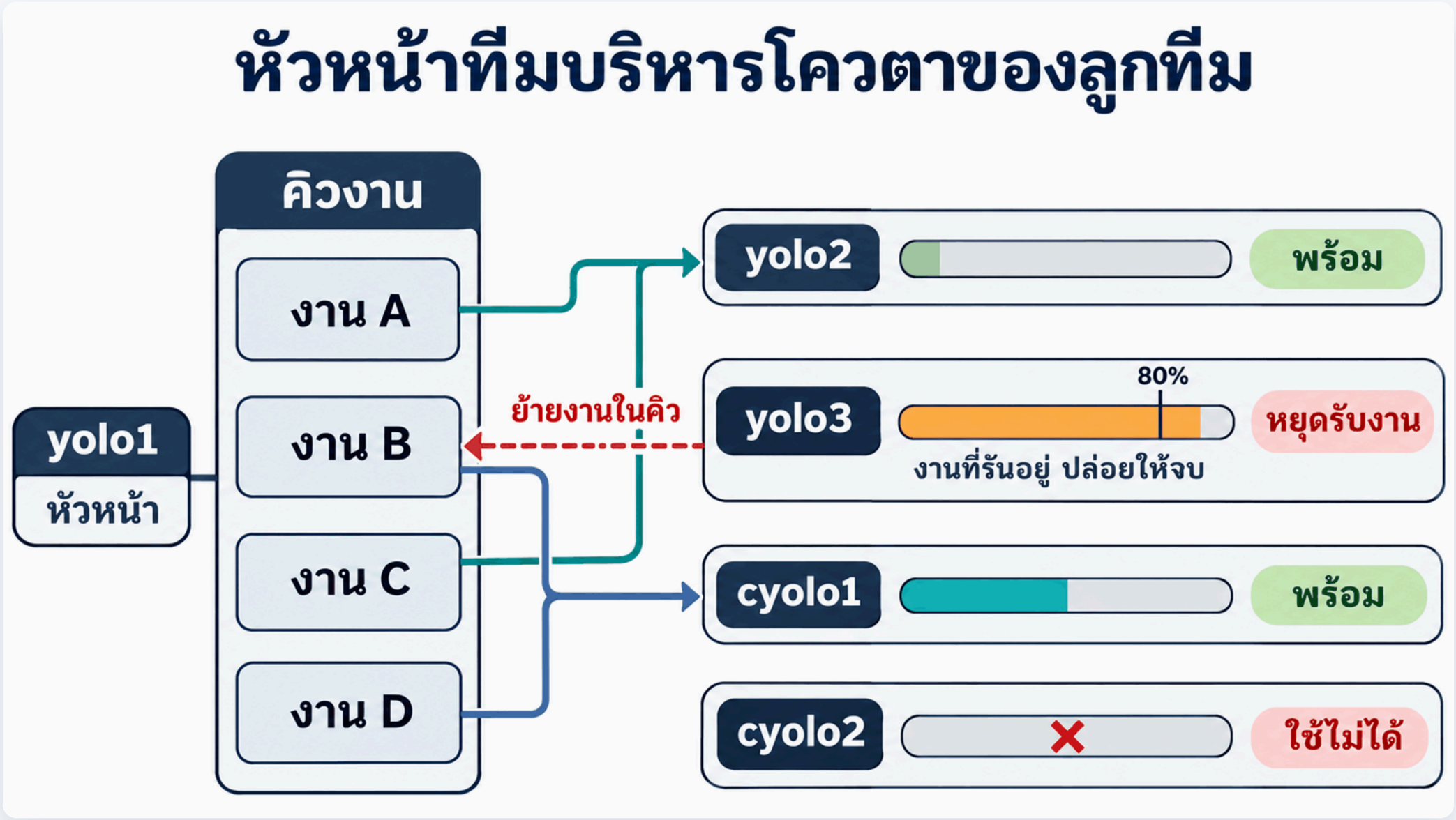
3.2 · รูปแบบขั้นสูง



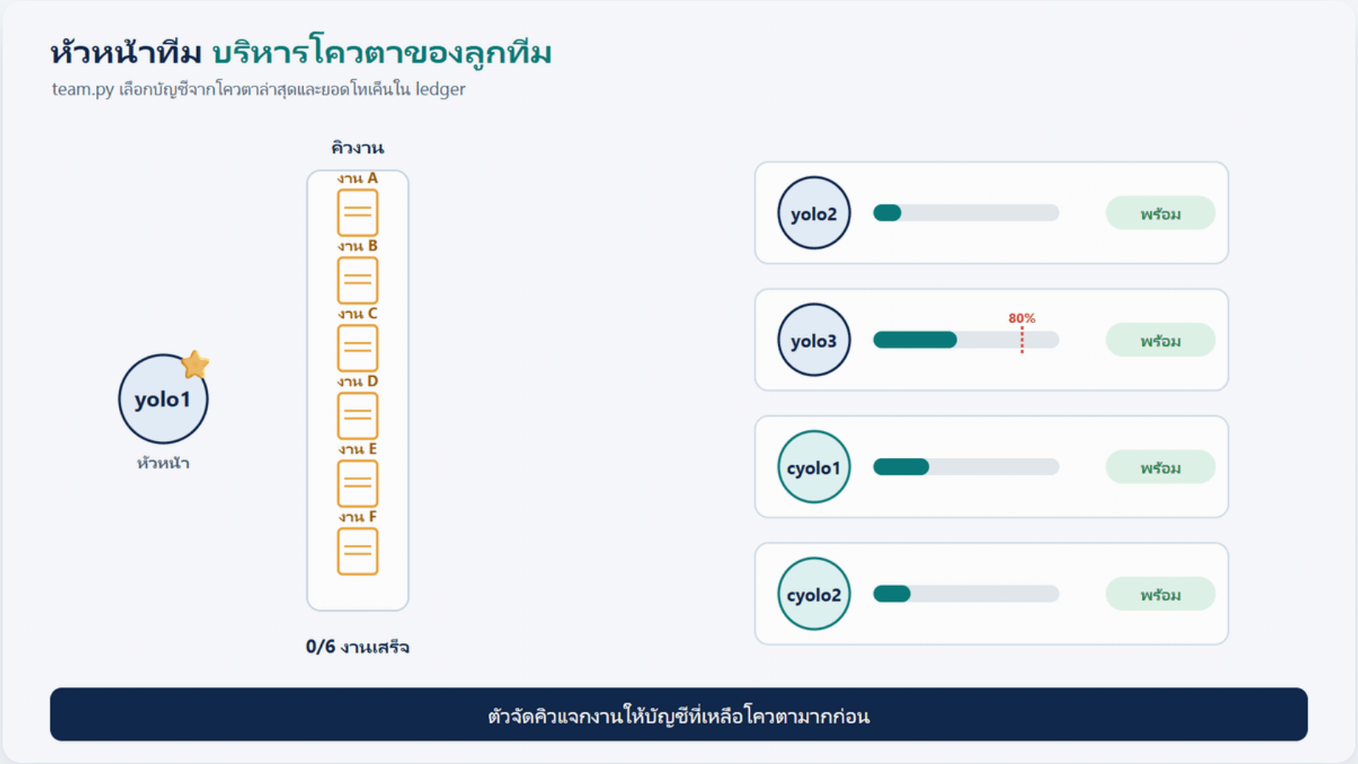
ต่อยอดได้: ทีมซ้อนทีม · โหวตเสียงข้างมาก · map-reduce · hedging · planner–executor–critic



- ทุกการเรียก = 1 บรรทัดใน `ledger.jsonl`
- `team.py stats` สรุปตามบัญชี / รุ่น / งาน
- `team.py quota` บอก**เวลาของข้อมูล**เสมอ
- **ไม่เก็บ prompt และ secret**


หัวหน้าทีมบริหารโควตาของลูกทีม
คิวงาน
งาน A
งาน B
งาน C
งาน D
yolo1
หัวหน้า
ย้ายงานในคิว
yolo2
พร้อม
yolo3
80%
หยุดรับงาน
งานที่รันอยู่ ปล่อยให้จบ
cyolo1
พร้อม
cyolo2
ใช้ไม่ได้

3.4 · หัวหน้าทีมบริหารโทเค็นของลูกทีม

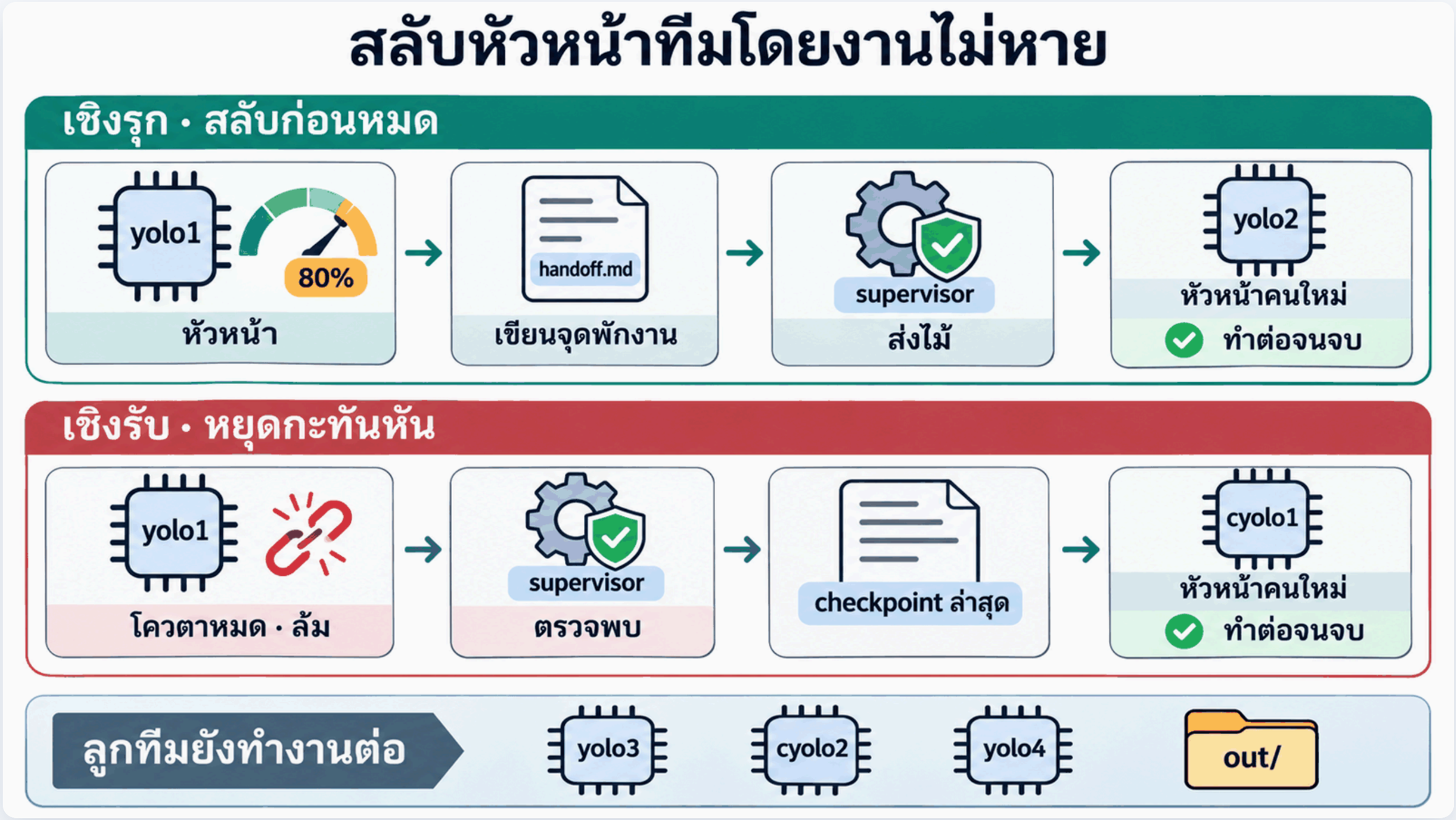


สถานการณ์	ตัวจัดคิวทำอะไร
ก่อนสั่งงาน	แจกให้บัญชีที่เหลือมากก่อน งานง่ายใช้รุ่นเล็ก
ใกล้หมด ≥ 80%	**หยุดรับงานใหม่** · งานในคิวย้าย
ล้มกลางคัน	ย้ายงาน + ความคืบหน้าใน msgs/
ผู้ใช้ระบุบัญชีเอง	ไม่ย้ายเงียบ ๆ หยุดและรายงาน

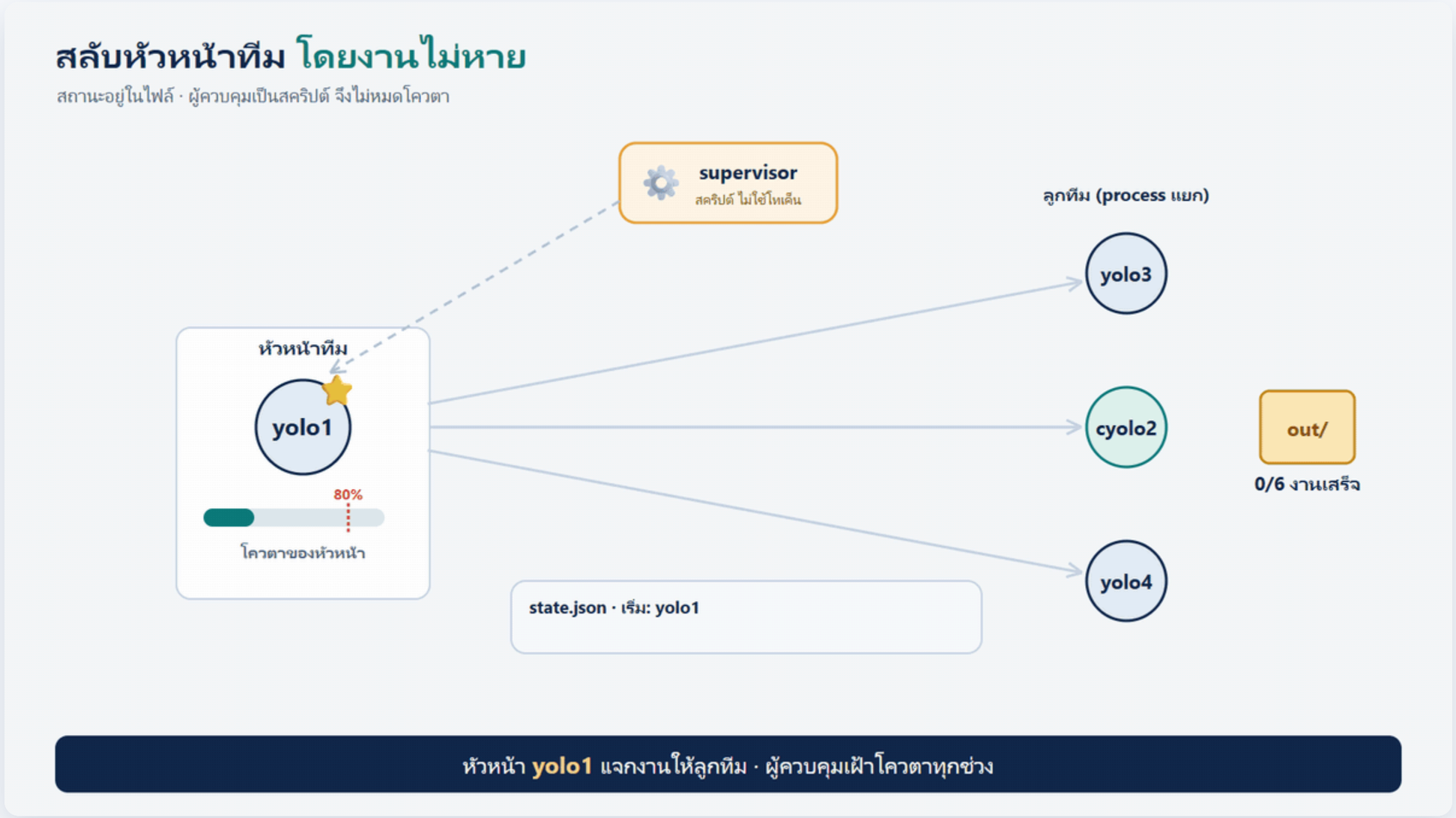


handoff.md = ใบส่งเวร

ก่อนออกเวรเขียนว่า ทำอะไรแล้ว · อะไรกำลังทำ · ต้องทำอะไรต่อ


สลับหัวหน้าทีมโดยงานไม่หาย
เชิงรุก · สลับก่อนหมด
yolo1
80%
หัวหน้า
handoff.md
เขียนจุดพักงาน
supervisor
ส่งไม้
yolo2
หัวหน้าคนใหม่
ทำต่อจนจบ
เชิงรับ · หยุดกะทันหัน
yolo1
โควตาหมด · ล้ม
supervisor
ตรวจพบ
checkpoint ล่าสุด
cyolo1
หัวหน้าคนใหม่
ทำต่อจนจบ
ลูกทีมยังทำงานต่อ
yolo3
cyolo2
yolo4
out/

3.5 · สลับหัวหน้าทีมโดยงานไม่หาย



3.5 · สลับหัวหน้าทีมโดยงานไม่หาย

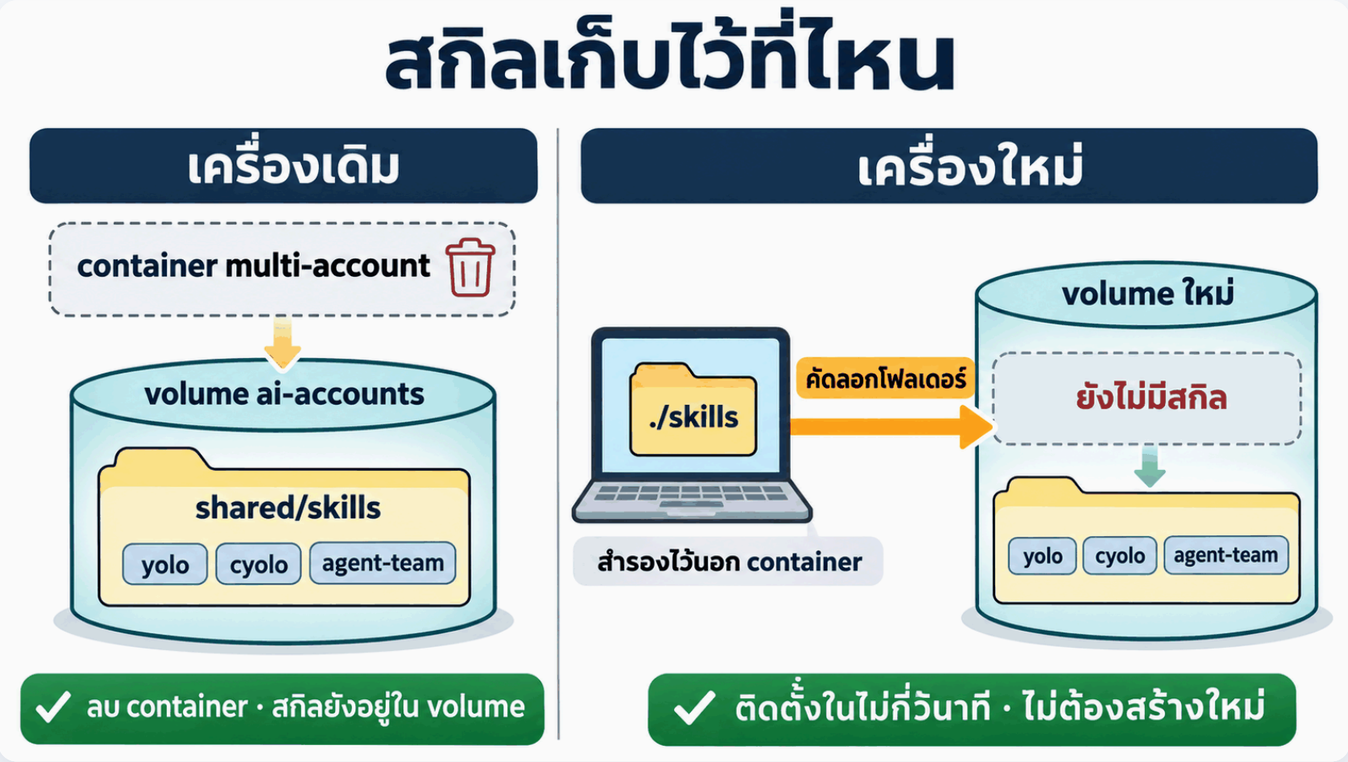
หลักการออกแบบ

หลักการ	รายละเอียด
สถานะอยู่ในไฟล์	state.json · handoff.md
ผู้ควบคุมเป็นสคริปต์	`team.py lead` ไม่ใช้โทเค็น · หัวหน้าทำงานเป็นรอบ เขียน checkpoint ทุกรอบ
ลูกทีมไม่ผูกกับหัวหน้า	process แยก · หัวหน้าเปลี่ยนตัว งานไม่หาย
สลับเชิงรุก	โควตา ≥ 80% หรือเกินงบ → ส่งไม้ตั้งแต่ยังเขียน checkpoint ทัน
สลับเชิงรับ	ล้ม · rate limit · เงียบไป → เริ่มคนใหม่จาก checkpoint
ผู้สืบทอด	พร้อม + โควตาเหลือมาก · ฝั่งเดียวกันก่อน (หัวหน้าเป็น Codex ได้)

ส่วนที่ 4 จาก 5

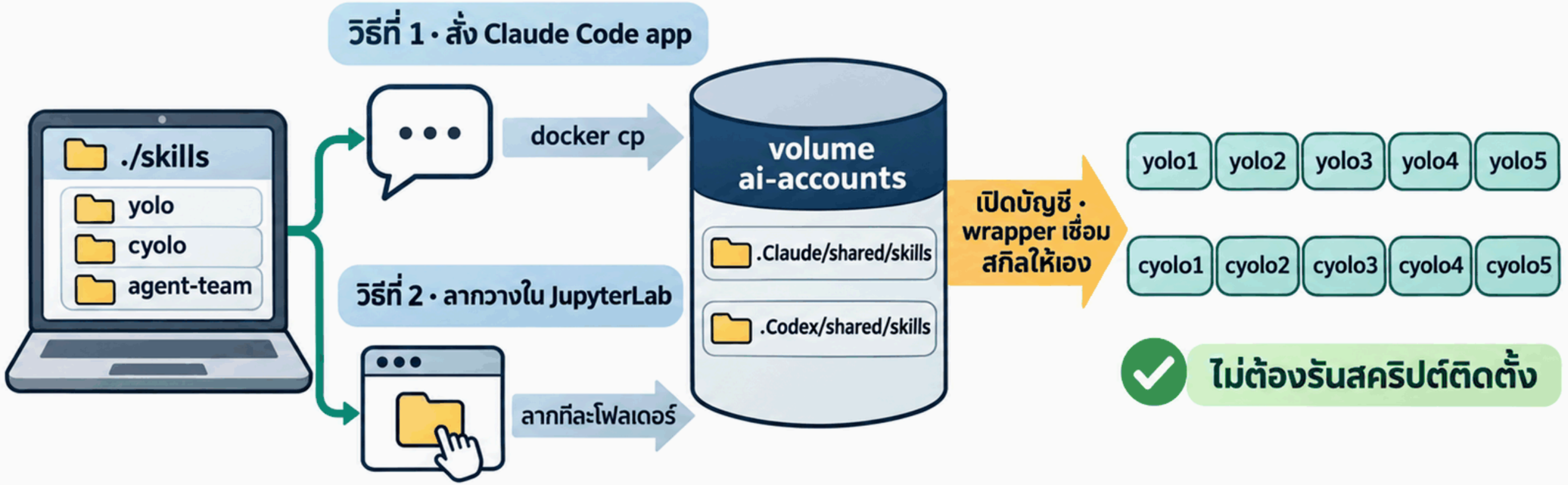
เก็บและติดตั้งสกิล

สกิลอยู่ใน volume ai-accounts · สำรองโฟลเดอร์ไว้นอก container · ติดตั้ง = คัดลอกไปวาง



ที่เก็บ	ลบ container	เครื่องใหม่
โฟลเดอร์สำรอง `skills/`	ยังอยู่	**นำไปติดตั้งได้**
`shared/skills` ใน volume	ยังอยู่	ไม่ติดไป
container	หายไป	ไม่ติดไป

ติดตั้งสกิล = คัดลอกโฟลเดอร์ 2 วิธี



ส่วนที่ 5 จาก 5

ลงมือ 4 ขั้น

สร้างสกิลจากศูนย์ด้วยสเปก + hint → ติดตั้ง → ทดสอบงานทีมชุดเดิม → สลับหัวหน้าและบริหารโทเค็น

5 · ภาพรวมขั้นตอนการทดลอง

ขั้น	ทำที่ไหน	เวลารันจริง
1 สร้างสกิลจากสเปก (จากศูนย์)	Claude Code app	~43 นาที
2 ติดตั้งด้วยการคัดลอกโฟลเดอร์	Claude Code app	67.9 วินาที
3 งานทีมชุดเดิมอีกครั้ง	Claude Code app	170.5 วินาที
4 สลับหัวหน้า + บริหารโทเค็น	Claude Code app	138.6 วินาที

```
04_MultiAccount_Skill/
├── prompt_skill_spec.md   ← สเปกของสกิล 3 ตัว: ต้องทำอะไร + เกณฑ์ทดสอบ (input ที่ 1)
├── prompt_skill_hints.md  ← ข้อเท็จจริงที่วัดแล้ว + กับดักจากการใช้งานจริง (input ที่ 2)
└── _solution/
    ├── skills/            ← สกิลที่ได้จากการรันจริง = โฟลเดอร์สำรองพร้อมติดตั้ง
    ├── TEST-REPORT.md     ← รายงานทดสอบของขั้นที่ 1
    └── BENCHMARK.md       ← ผลวัด: สกิลจากศูนย์เทียบกับสกิลชุดเดิม
```

ต้องมี container multi-account และล็อกอินอย่างน้อย 1 บัญชี Claude + 1 บัญชี Codex

1

ให้ Claude สร้างสกิลจากสเปก

สร้าง**จากศูนย์** ไม่มีสกิลเก่าให้ลอก · input: **prompt_skill_spec.md** + **prompt_skill_hints.md** → **./skills** (~43 นาที)

ขั้นที่ 1 · input 2 ไฟล์ · ไม่มีสกิลเก่า

prompt_skill_spec.md · ต้องทำอะไร

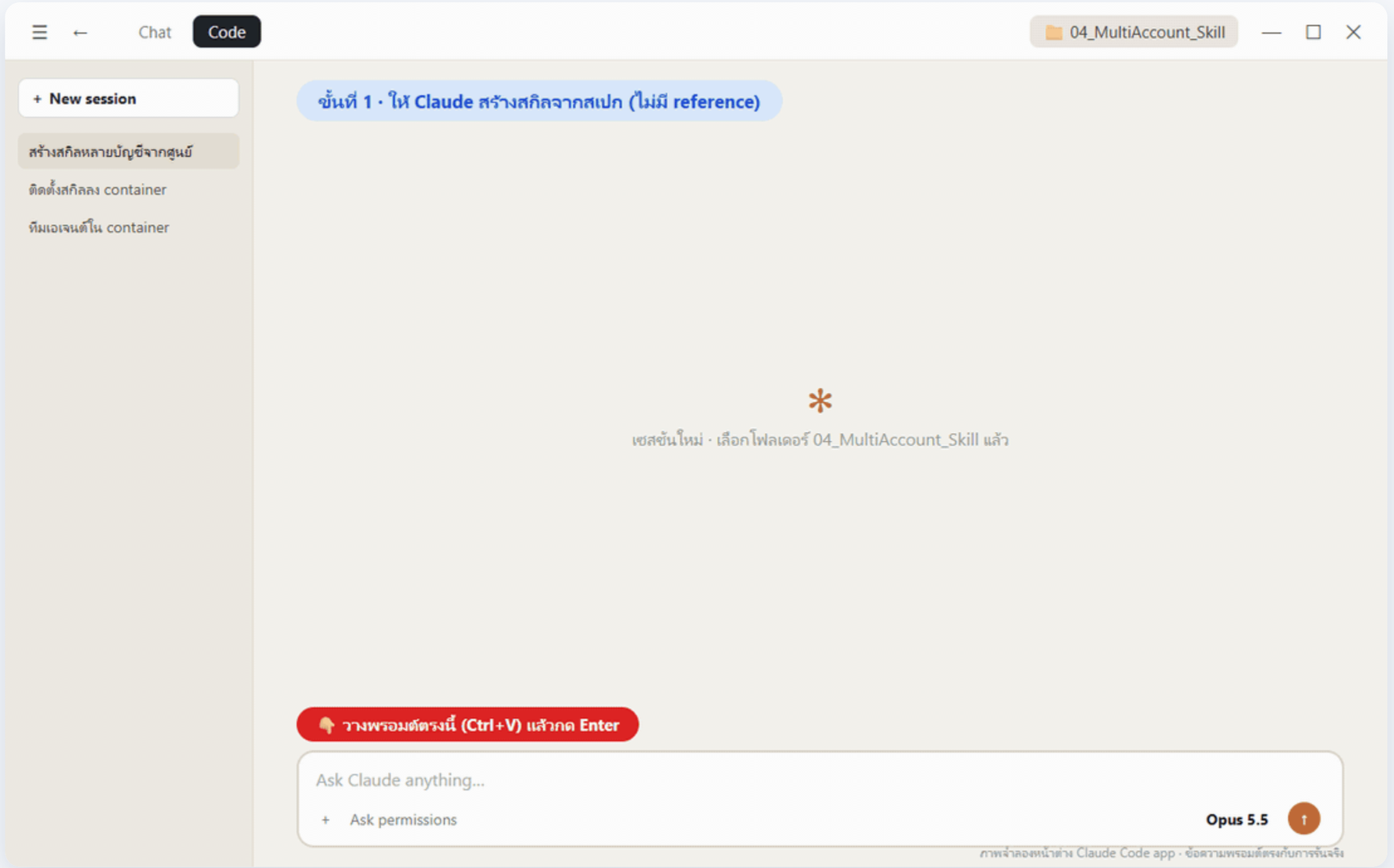
สกิล 3 ตัว · รูปแบบทำงานร่วมกัน · บริหารโทเค็น · สลับหัวหน้า · ledger · ใช้งานจาก host · ติดตั้งด้วยการคัดลอก · เกณฑ์ทดสอบ

prompt_skill_hints.md · รู้อะไรแล้วบ้าง

ข้อเท็จจริงของ CLI ที่วัดแล้ว (flag · JSON · โควตา · เวลา) + กับดักที่เห็นจากการที่ Claude Code app ใช้สกิลจริง

หัวข้อ	สาระ
ข้อเท็จจริงจากการทดลอง (hint)	ปิด stdin · exit code ไม่ใช่ตัวบอกผล · auth status ไม่ตรงจริง · /usage · rollout · ephemeral เร็วกว่า 3.5 เท่า
บทเรียนจากการใช้งานจริง (hint)	หา container · path ใน Git Bash · expect เข้มเกิน · ต้องเปิดโค้ดเพื่อหาวิธีสาธิต → สิ่งที่สกิลควรทำ
agent-team (สเปก)	run directory · พื้นฐาน 5 แบบ · ขั้นสูง 5 แบบ · บริหารโทเค็น · สลับหัวหน้า · สาธิตคำสั่งเดียว
ใช้งานให้จบในไม่กี่คำสั่ง (สเปก)	กราฟตัวอย่าง · run จบด้วยตาราง + โควตา · --resume · ใช้ได้ทั้ง PowerShell และ Git Bash
ทดสอบ (สเปก)	ทุกรูปแบบด้วยบัญชีจริง + ทดสอบแบบผู้ใช้จริง 3 โจทย์

ขั้นที่ 1 · 💻 Claude Code app



ขั้นที่ 1 · พรอมต์ (คัดลอกทั้งกรอบ)

```
ช่วยสร้างสกิลชุดใหม่ที่ทำให้ AI ใช้ระบบหลายบัญชีใน container multi-account ได้เต็มประสิทธิภาพ
ทำในโฟลเดอร์นี้ ใช้คำสั่ง docker ได้ แต่ห้ามลบ หยุด หรือสร้าง container / volume ใหม่ ห้ามแตะ credential และห้ามล็อกอินแทนผม

วัตถุดิบ
- ./prompt_skill_spec.md = สเปกของสกิล 3 ตัว (yolo, cyolo, agent-team) ทุกข้อต้องเป็นจริง
- ./prompt_skill_hints.md = ข้อเท็จจริงที่วัดมาแล้วและกับดักที่พบจากการใช้งานจริง ใช้แทนการลองผิดลองถูก
- container multi-account ที่กำลังรัน = ที่ทดสอบจริง (บัญชีที่พร้อมอาจมีไม่ครบ ให้ตรวจเอง)

สร้างจากศูนย์ไว้ใน ./skills (นอก container นี่คือต้นฉบับที่เก็บสำรอง)
ทดสอบ: คัดลอกโฟลเดอร์สกิลใน ./skills ไปวางที่ shared ของทั้งสองฝั่งใน container แล้วทดสอบทุกข้อในหัวข้อ "ทดสอบ" ของสเปก
ใช้บัญชีอย่างประหยัด (haiku/low กับงานทดสอบ) ข้อที่ต้องใช้บัญชีที่ไม่พร้อม = SKIP
เขียน ./TEST-REPORT.md (นอกโฟลเดอร์ skills) แล้วสรุปเป็นตารางสั้น ๆ: ผลทดสอบ ไฟล์ที่สร้าง และเวลาที่ใช้ แล้วหยุด
```

ขั้นที่ 1

หลักการออกแบบพรอมต์

แยก "ต้องทำอะไร" กับ "รู้อะไรแล้ว"

สเปกบอกเป้าหมาย + เกณฑ์ทดสอบ · hint บอกข้อเท็จจริงที่วัดแล้ว ไม่ต้องมีสกิลเก่าเป็นต้นแบบ

hint มาจากการใช้งานจริง

ดูว่า Claude Code app ติดตรงไหนเมื่อใช้สกิล แล้วเขียนเป็น "สิ่งที่ทำให้ช้า → สิ่งที่สกิลควรทำ"

ต้นฉบับอยู่นอก container

./skills คือโฟลเดอร์สำรองที่นำไปติดตั้งเครื่องใหม่ได้

ทดสอบแบบประหยัด

งานทดสอบใช้รุ่นเล็ก และจำลองโควตาหมดแทนการใช้จริง

ขั้นที่ 1 · ผลจริง 8 พ.ย. 2026 · ~43 นาที · 93 คำสั่ง

PASS 21 · SKIP บางส่วน 1

✻ **Claude** · ขั้นที่ 1 · ให้ Claude สร้างสกิลจากสเปก (ไม่มี reference) · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

สร้างสกิลครบทั้ง 3 ตัวและทดสอบทุกข้อในหัวข้อ "ทดสอบ" ของสเปกใน container `multi-account` แล้ว ผลคือ **PASS 21 ข้อ · SKIP บางส่วน 1 ข้อ** ใช้เวลารวมประมาณ 43 นาที ไม่ได้สร้าง หยุด หรือลบ container/volume ไม่ได้แตะ credential และไม่ได้ล็อกอินแทน

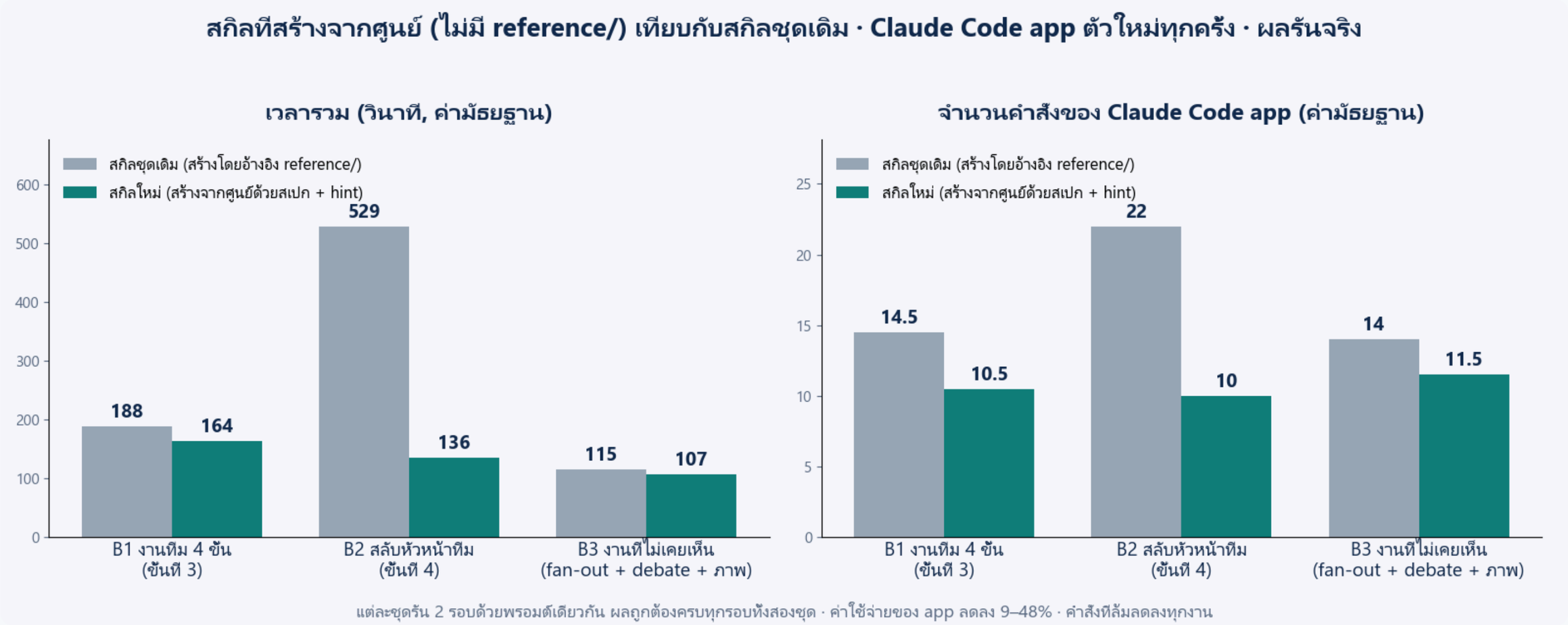
หัวข้อ	ผล	หลักฐาน / เวลา
ทุกบัญชีมองเห็นสกิลทั้ง 3	PASS	10 บัญชี ครบ 30/30
ตรวจความพร้อม แยก 3 สถานะ	PASS	พร้อม / หมดอายุ / ไม่ได้ล็อกอิน · 6 s
เรียกงานเดียว + แถวใน ledger	PASS	ทั้งสองฝั่งได้ฟิลด์ครบ
relay ข้าม CLI	PASS	42 → 142 → 426 · 12 s
fan-out/fan-in, `{{file:…}}`, review loop, board, debate, best-of, router	PASS	กราฟเดียว 18 งาน สำเร็จ 18/18 ใน 27 s
ผลรวมสถิติตรงกับ ledger	PASS	ตรงทุกค่า
failover (`MA_SIMULATE_LIMIT`)	PASS	ย้ายงานไปบัญชีอื่น · งานที่ระบุบัญชีเองหยุดและรายงาน
human gate, `--resume`, งบโทเค็นลูกทีม	PASS	`status` แสดง "หยุดรับงาน"
สลับหัวหน้าเชิงรับ + เชิงรุก + ย้ายงานลูกทีม ในคำสั่งเดียว	PASS	สูตรคูณถูก 96/96 ใน 59 s · กับ `--goal` ใหม่ 69 s
เรียกจาก host ทุกบรรทัดใน SKILL.md	PASS	Git Bash ทุกบรรทัด exit 0 · PowerShell ผ่าน
สร้างภาพ PNG ผ่าน cyolo	PASS	PNG ถูกต้อง · 27 s
ทดสอบแบบผู้ใช้จริง (ก)(ข)(ค)	PASS หลังปรับสกิล	รอบสุดท้ายไม่มีคำสั่งล้ม · 2–3 คำสั่งต่อโจทย์ · ไม่เปิดโค้ดสกิล
`auth status` บอกว่าล็อกอิน แต่ token หมดอายุ	SKIP บางส่วน	ไม่มีบัญชีในสถานะนี้ให้ทดสอบ

ทดสอบแบบผู้ใช้จริงเจอปัญหาจากสกิล 3 จุด แก้แล้วทุกจุด: ระบุบัญชีที่ไม่พร้อมแล้วงานล้มกลางทาง → `run` ไม่เริ่มและบอกบัญชีที่พร้อม · worker ตอบแค่บรรทัด `BOARD:` → ใช้เป็นคำตอบด้วย · หัวหน้าส่ง `difficulty: "easy"` → แปลงค่าให้เอง

ไฟล์ที่สร้าง: `./skills/` (yolo · cyolo · agent-team) · `./TEST-REPORT.md`

คำตอบจริง จัดหน้าใหม่ · ตลอดการรันไม่ได้เปิดสกิลชุดเก่าเลย · SKIP = ไม่มีบัญชีที่ auth status บอกว่าล็อกอินแต่ token หมดอายุ

1.4 · พิสูจน์ · สกิลจากศูนย์ vs สกิลชุดเดิม



แต่ละชุดรัน 2 รอบด้วยพรอมต์เดียวกัน ผลถูกต้องครบทุกรอบทั้งสองชุด · ค่าใช้จ่ายของ app ลดลง 9–48% · คำสั่งทีล้มลดลงทุกงาน

1.4 · พิสูจน์ · ค่ามัธยฐาน 2 รอบ · Claude Code app ตัวใหม่ทุกครั้ง

ดีกว่าเดิมทุกงาน ผลถูกครบทุกรอบ

งาน	เวลา (วินาที)	คำสั่งของ app	คำสั่งที่ล้ม	ค่าใช้จ่าย
B1 งานทีม 4 ขั้น (ขั้นที่ 3)	188.1 → **164.1**	14.5 → **10.5**	3.5 → **2**	$0.68 → **$0.47**
B2 สลับหัวหน้าทีม (ขั้นที่ 4)	528.6 → **135.5**	22 → **10**	5 → **2.5**	$0.97 → **$0.50**
B3 งานที่ไม่เคยเห็น	115.1 → **107.4**	14 → **11.5**	5 → **3**	$0.58 → **$0.53**

วนปรับ 2 รอบ: รอบแรกผ่านแบบหวุดหวิด → เพิ่มสิ่งที่เห็นลงสเปก + hint แล้วสร้างใหม่จากศูนย์ · รอบที่ 2 ดีกว่าเดิมทุกงาน · รายละเอียดใน _solution/BENCHMARK.md

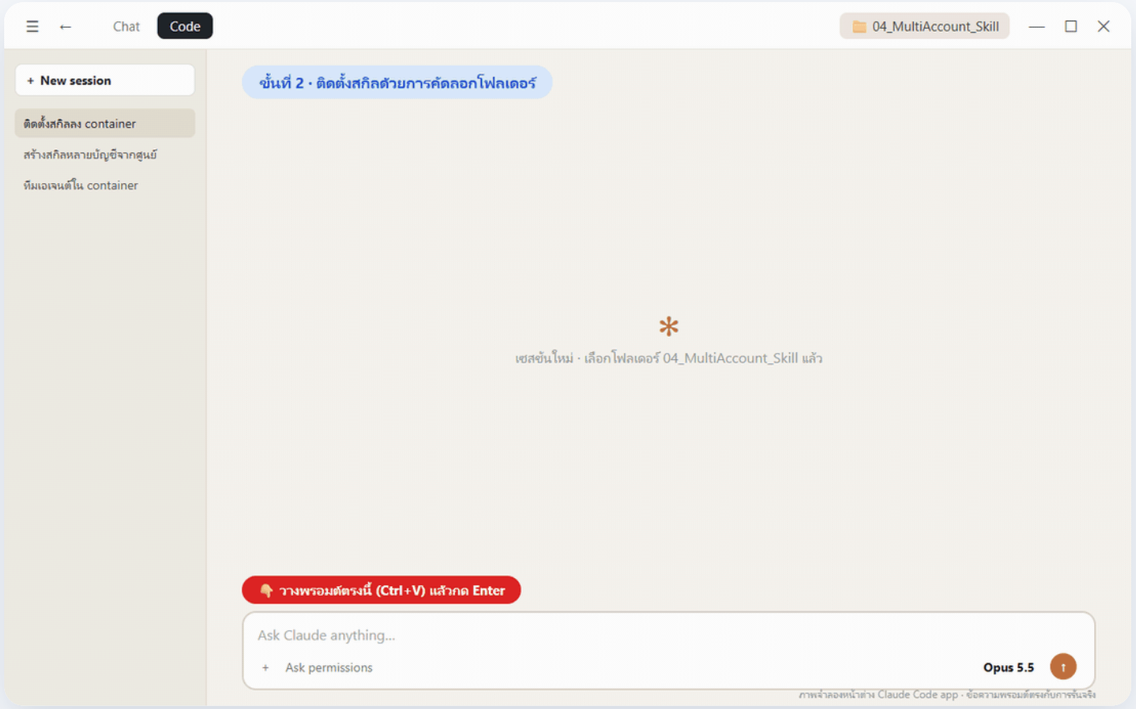
2

ติดตั้งสกิลด้วยการคัดลอกโฟลเดอร์

คัดลอก **yolo** · **cyolo** · **agent-team** ลง shared/skills · ไม่ต้องรันสคริปต์ติดตั้ง (67.9 วินาที)

ขั้นที่ 2 · วิธีที่ 1 · 💻 Claude Code app

ติดตั้งสกิลจาก ./skills ลง container multi-account ด้วยการคัดลอกโฟลเดอร์อย่างเดียว
- คัดลอกโฟลเดอร์ yolo, cyolo และ agent-team ใน ./skills ไปวางที่ /root/ai-accounts/.Claude/shared/skills และ /root/ai-accounts/.Codex/shared/skills ใน container
- เปิดแต่ละบัญชีหนึ่งครั้งด้วย yoloN --version และ cyoloN --version (wrapper จะเชื่อมสกิลให้แต่ละบัญชีเอง)
- ตรวจว่าทุกบัญชี yolo1-5 และ cyolo1-5 มองเห็น SKILL.md ของทั้ง 3 สกิล และตรวจความพร้อมด้วยคำสั่ง check ของสกิล agent-team
สรุปเป็นตาราง สกิล | ฝั่ง Claude | ฝั่ง Codex แล้วหยุด



ขั้นที่ 2 · ผลจริง

✻ **Claude** · ขั้นที่ 2 · ติดตั้งสกิลด้วยการคัดลอกโฟลเดอร์ · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

ติดตั้งสกิลเสร็จแล้วใน container `multi-account` โดยการคัดลอกโฟลเดอร์อย่างเดียว (`docker cp` 6 ครั้ง) ทุกบัญชีมองเห็นสกิลครบทั้ง 3 ตัว

สกิล	ฝั่ง **Claude** (`.Claude/shared/skills`)	ฝั่ง **Codex** (`.Codex/shared/skills`)
yolo	✅ yolo1–5 เห็น SKILL.md	✅ cyolo1–5 เห็น SKILL.md
cyolo	✅ yolo1–5 เห็น SKILL.md	✅ cyolo1–5 เห็น SKILL.md
agent-team	✅ yolo1–5 เห็น SKILL.md	✅ cyolo1–5 เห็น SKILL.md

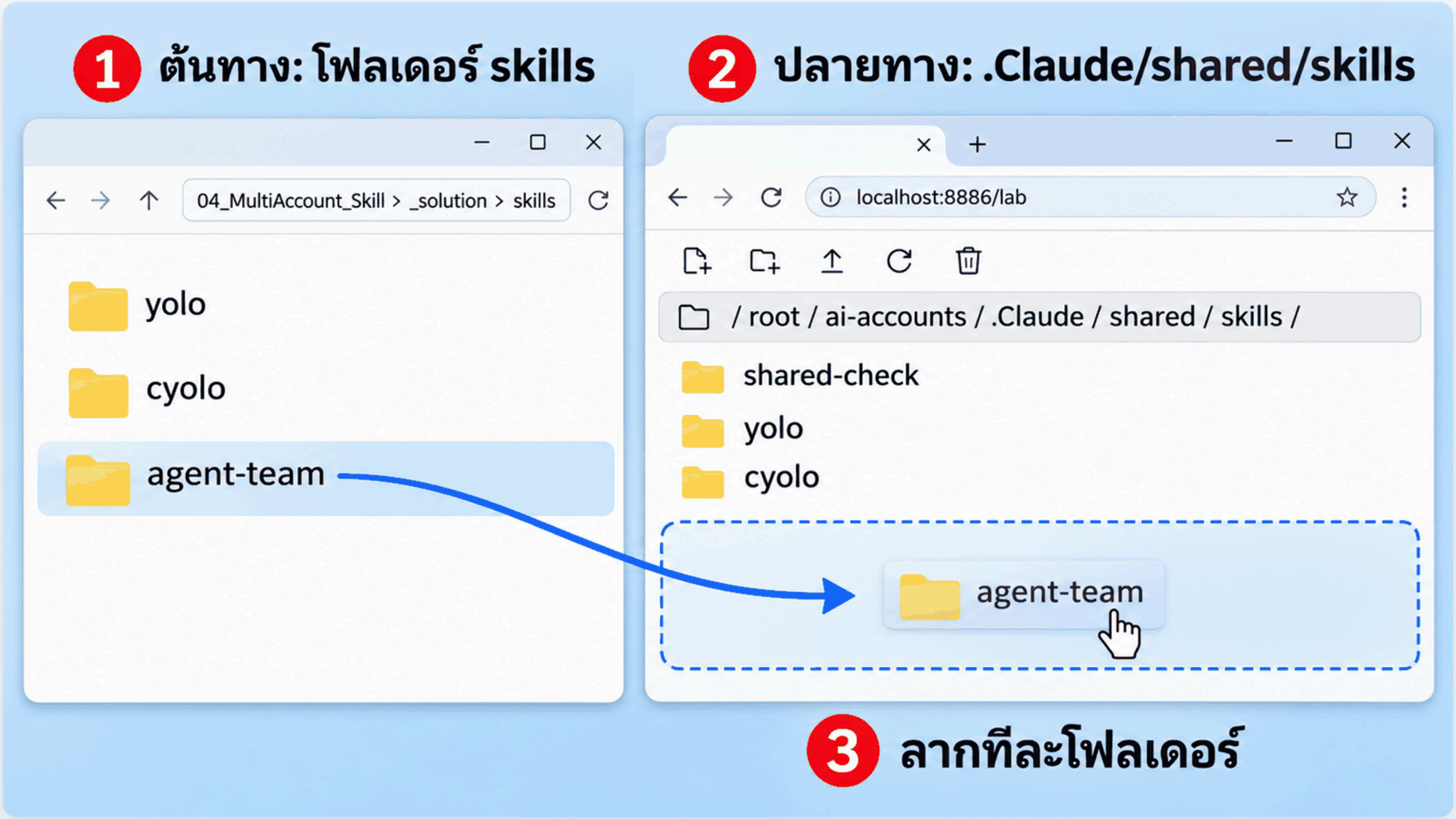
- **เปิดบัญชี:** `yoloN --version` ทั้ง 5 บัญชีได้ `2.1.319 (Claude Code)` ส่วน `cyoloN --version` ได้ `codex-cli 0.167.0` ทุกบัญชี
- **การเชื่อมสกิล:** ฝั่ง Claude โฟลเดอร์ `skills` ของแต่ละบัญชีชี้ไปที่ `../shared/skills` ทั้งโฟลเดอร์อยู่แล้ว ฝั่ง Codex ตัว wrapper สร้างลิงก์ `yolo` , `cyolo` , `agent-team` ให้แต่ละบัญชีตอนเปิดครั้งนี้
- **`team.py check` :** เรียกจริงทุกบัญชีและแยกสถานะ พร้อม / token หมดอายุ / ไม่ได้ล็อกอิน

หยุดตามที่สั่งไว้

> คัดลอกด้วย docker cp 6 ครั้ง · เปิดบัญชีด้วย --version แล้ว wrapper เชื่อมสกิลให้เอง · ทุกบัญชีเห็น SKILL.md ของทั้ง 3 สกิล

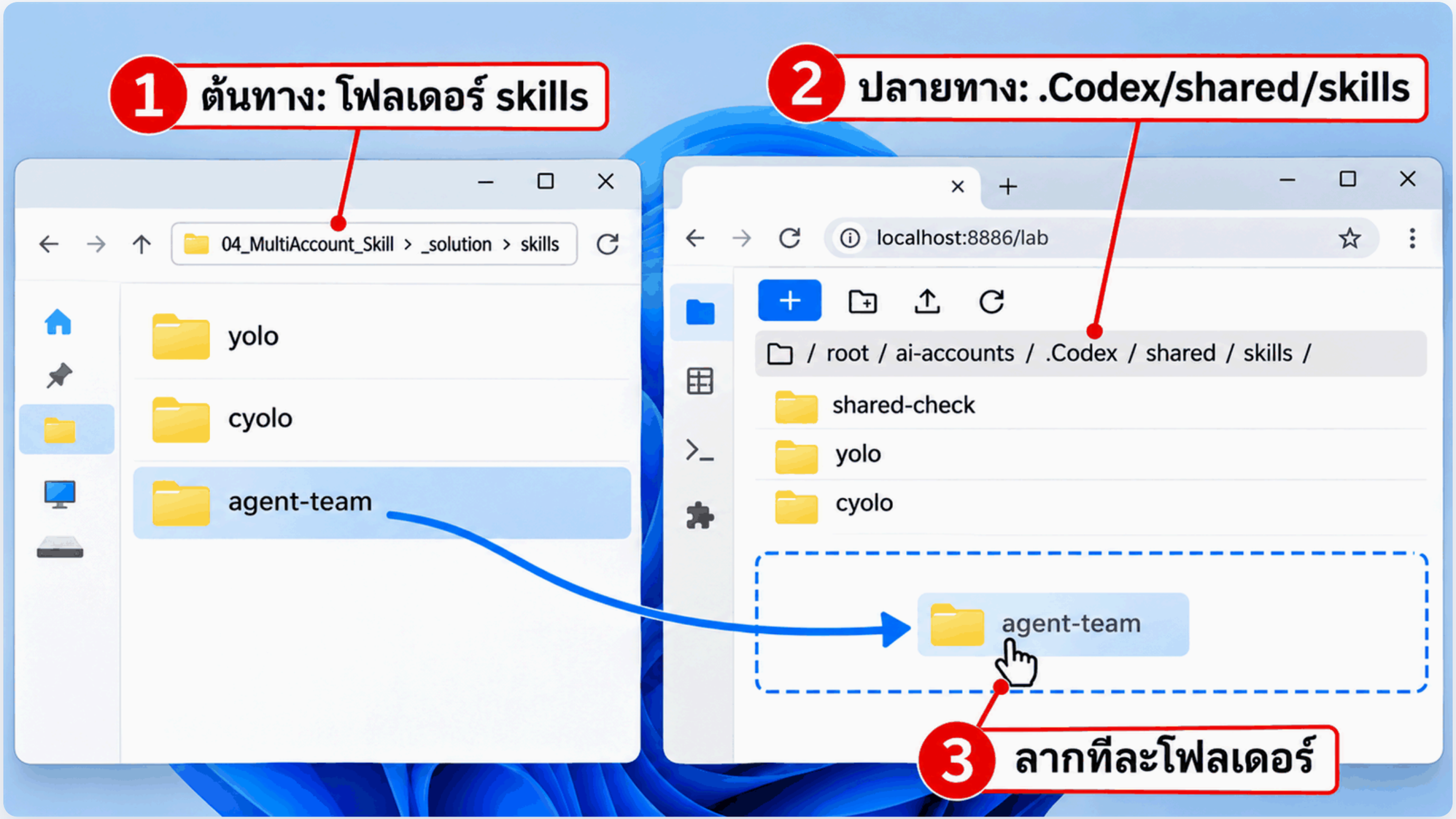
ใช้สกิลที่เตรียมไว้: เปลี่ยน ./skills เป็น ./_solution/skills

ขั้นที่ 2 · วิธีที่ 2 · ลากวางใน JupyterLab · ปลายทางที่ 1



http://localhost:8886/lab/tree/root/ai-accounts/.Claude/shared/skills · ใช้กับ yolo1–5

ขั้นที่ 2 · วิธีที่ 2 · ลากวางใน JupyterLab · ปลายทางที่ 2



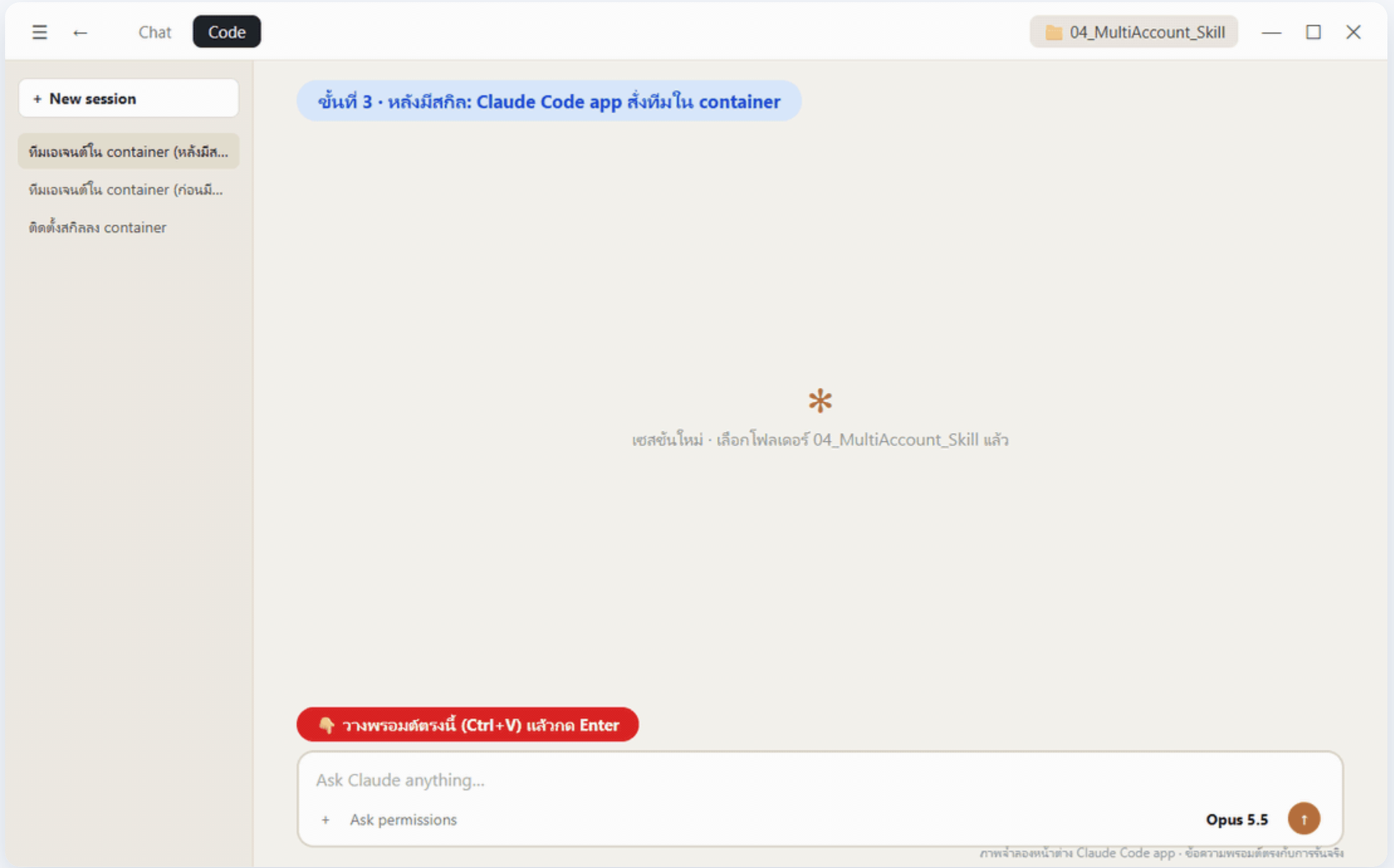
http://localhost:8886/lab/tree/root/ai-accounts/.Codex/shared/skills · ใช้กับ cyolo1–5 · ลากครบแล้วเปิด yolo1 และ cyolo1 หนึ่งครั้ง

3

ทดสอบงานทีมชุดเดิมอีกครั้ง

วางพรอมต์ชุดเดียวกับหัวข้อ 1.1 ใน **Claude Code app** · ตอนนี้ใน container มีสกิลแล้ว (170.5 วินาที)

ขั้นที่ 3 · 💻 Claude Code app · มีสกิลแล้ว



ขั้นที่ 3 · ผลจริงหลังมีสกิล

✻ **Claude** · ขั้นที่ 3 · หลังมีสกิล: Claude Code app สั่งทีมใน container · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

งานทั้ง 4 ขั้นเสร็จครบ และ `final.py` ผ่านทดสอบทั้ง 5 กรณี ผมรันชุดทดสอบซ้ำใน container เองด้วย ได้ `Ran 5 tests … OK`

ใช้ container `multi-account` เพราะมีสกิล `agent-team` ผมอ่านสกิลนี้แล้วใช้คำสั่ง `team.py run` รันลำดับงาน run id คือ `20261108-120428-ee1a`

ผลแต่ละขั้น (ไฟล์อยู่ที่ `/workspace/team-demo`)

1. yolo1 เขียน `a_claude.py` และ cyolo1 เขียน `a_codex.py` พร้อมกัน
2. cyolo2 ตรวจทั้งสองไฟล์และเลือก **a_codex.py** เพราะสั้นกว่า และใช้ `re.fullmatch` ตรวจอินพุตเข้มกว่า จุดที่ให้แก้: จำกัดความยาวอินพุต · เขียนให้ชัดว่ายอมรับ `60m` / `90s` หรือไม่ · รับเฉพาะเลข ASCII · เพิ่มการทดสอบกรณีขอบ
3. yolo2 แก้เป็น `final.py` และเขียน `test_final.py` ทดสอบ 5 กรณี: `1h30m` → 5400 · `45s` → 45 · `2h` → 7200 · `1h2m3s` → 3723 · `abc` → ValueError

4. สถิติ (ค่าที่บันทึกใน ledger ของแต่ละบัญชีระหว่างรัน)

บัญชี	งาน	เวลา (วินาที)	input tokens	output tokens	สำเร็จ/ล้ม
yolo1 (Claude sonnet)	เขียน a_claude.py	7.7	56,978	620	สำเร็จ
cyolo1 (Codex gpt-6.1-sol)	เขียน a_codex.py	22.9	49,113	775	สำเร็จ
cyolo2 (Codex gpt-6.1-sol)	ตรวจและเลือกฉบับที่ดีกว่า	28.7	58,644	938	สำเร็จ
yolo2 (Claude sonnet)	แก้เป็น final.py + ทดสอบ 5 กรณี	14.8	112,828	1,817	สำเร็จ (PASS)
รวม		**74.1**	**277,563**	**4,150**	**4/4**

สองงานแรกรันพร้อมกัน เวลาจริงตั้งแต่ต้นจนจบจึงประมาณ 69 วินาที

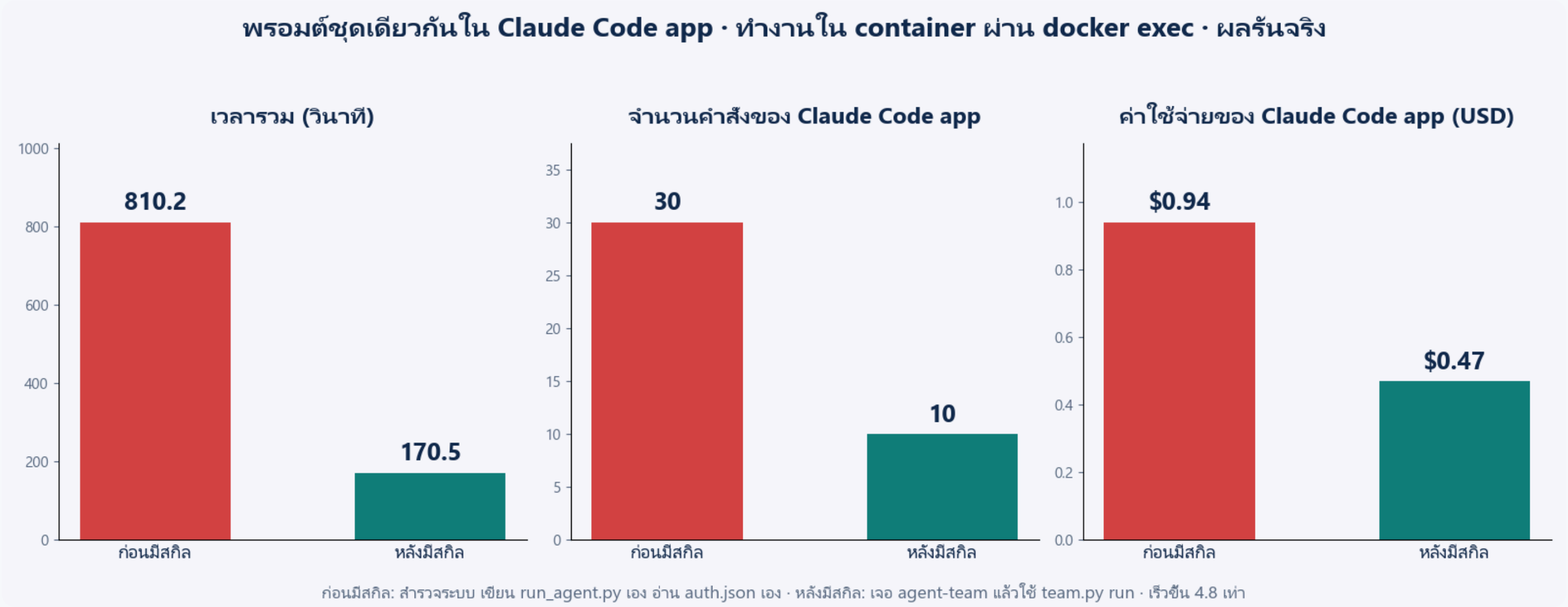
5. โควตาที่เหลือ

บัญชี	ใช้ไป	เหลือ	รีเซ็ต
yolo1	week 23%	**77%**	12 พ.ย. 05:59
yolo2	session 2% · week 3%	**session 98% · week 97%**	session 8 พ.ย. 16:09 · week 9 พ.ย. 12:59
cyolo1	primary (รอบ 7 วัน) 3%	**97%**	14 พ.ย. 12:21
cyolo2	primary (รอบ 7 วัน) 3%	**97%**	14 พ.ย. 12:21

> ตรวจโฟลเดอร์สกิลแล้ว**เจอ agent-team** · อ่านแค่ SKILL.md · team.py check · เขียน graph.json แล้ว team.py run ครั้งเดียวได้ผลงาน + ตารางจาก ledger + โควตา

คำตอบจริง ตัดรายชื่อบัญชีที่ล็อกอินบนเครื่องผู้สอนออก

ขั้นที่ 3 · ก่อน vs หลังมีสกิล



ขั้นที่ 3 · ก่อน vs หลังมีสกิล

เร็วขึ้น 4.8 เท่า

	ก่อนมีสกิล	หลังมีสกิล
เวลารวม	810.2 วินาที	**170.5 วินาที**
คำสั่งของ Claude Code app	30 (ล้ม 9)	10 (ล้ม 2)
ค่าใช้จ่าย	$0.94	$0.47
วิธีเรียกแต่ละบัญชี	เขียน run_agent.py เอง	team.py run ของสกิล
ไฟล์ credential	ตรวจ .credentials.json · เปิด auth.json	ไม่แตะ
โทเค็นและโควตา	สคริปต์ที่เขียนเอง	ตารางจาก ledger + โควตา ใน team.py run

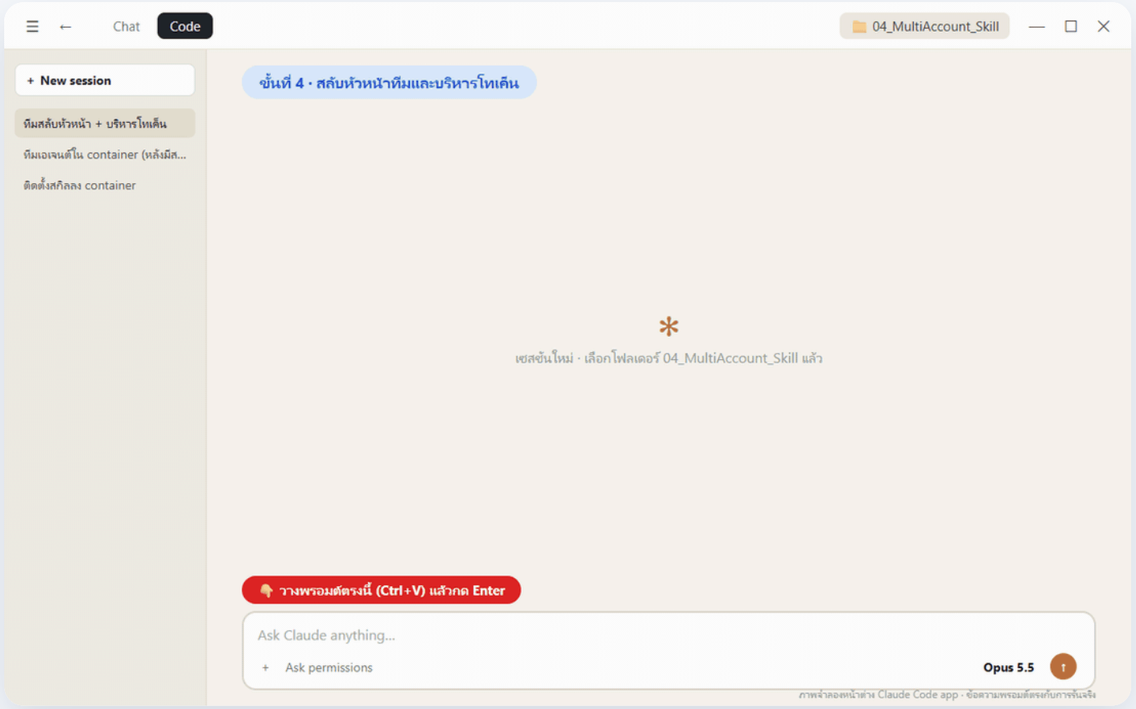
4

สลับหัวหน้าทีมและบริหารโทเค็น

team.py lead + ตัวจำลอง: หัวหน้าล้ม · หัวหน้าเกินงบ · ลูกทีมเกินงบ ในคำสั่งเดียว (138.6 วินาที)

ขั้นที่ 4 · 💻 Claude Code app

```
ใช้ทีมเอเจนต์ใน container multi-account ทดลองการสลับหัวหน้าทีมและการบริหารโทเค็นของลูกทีม
งานของทีม: สร้างตารางสูตรคูณแม่ 2 ถึงแม่ 9 แบ่งให้ลูกทีมทำคนละแม่ แล้วรวมเป็นไฟล์เดียว
/workspace/demo/table.md
- ใช้คำสั่ง lead ของสกิล agent-team งานทุกชิ้นใช้ความยาก 1
- ตั้งงบโทเค็นของหัวหน้าให้ต่ำ เพื่อให้เกิดการสลับหัวหน้าแบบเชิงรุกอย่างน้อย 1 ครั้ง
- จำลองให้หัวหน้าคนหนึ่งล้มกลางคัน เพื่อดูการสลับแบบเชิงรับ
- ตั้งงบโทเค็นของลูกทีมคนหนึ่งให้ต่ำ เพื่อดูการย้ายงานไปลูกทีมคนอื่น
สรุปเป็นตาราง: ประวัติการสลับหัวหน้า (จากใคร ไปใคร เพราะอะไร) การย้ายงานของลูกทีม ผลงานครบหรือไม่ และ
สถิติแยกตามบัญชี
```



ขั้นที่ 4 · ผลจริง · app 10 คำสั่ง · run 69 วินาที · 11 การเรียก

เหตุการณ์	เกิดอะไรขึ้น
สลับ**เชิงรับ**	yolo2 ล้ม (--simulate-crash) → yolo1 ทำต่อจาก state.json · งานที่สั่งแล้วไม่ถูกเพิ่มซ้ำ
สลับ**เชิงรุก**	yolo1 ใช้ 135,098 โทเค็น เกินงบ → checkpoint → cyolo1 (หัวหน้าฝั่ง Codex) ปิดงาน
ย้ายงานลูกทีม	cyolo2 เกินงบหลังทำ t4 → หยุดรับงาน · t8 ย้ายไป cyolo1

ตารางสูตรคูณครบ 8 แม่ 96 แถว ถูกทุกแถว · งบ 1 โทเค็นคือสวิตช์จำลอง

✻ **Claude** · ขั้นที่ 4 · สลับหัวหน้าทีมและบริหารโทเค็น · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

รันเสร็จแล้ว (run `mul-lead-demo` ใช้เวลา 69 วินาที) ทั้งสามเหตุการณ์ที่ต้องการเกิดขึ้นครบ และได้ไฟล์ `/workspace/demo/table.md`

ตั้งค่า: ลำดับหัวหน้า yolo2 → yolo1 → cyolo1 · yolo2 ถูกจำลองให้ล้ม (`--simulate-crash yolo2`) · yolo1 งบหัวหน้า 1 โทเคน · cyolo2 งบลูกทีม 1 โทเคน · ทุกงานความยาก 1 และ `--per-account 1`

ประวัติการสลับหัวหน้า

#	จาก	ไป	ชนิด	เหตุผล
1	yolo2	yolo1	เชิงรับ	หัวหน้าถูก kill หลังแจกงานครั้งแรก (จำลอง) คนใหม่ทำต่อจาก `state.json`
2	yolo1	cyolo1	เชิงรุก	หัวหน้าใช้ไป 135,098 โทเคน เกินงบ 1 เขียน checkpoint แล้วส่งไม้

การย้ายงานของลูกทีม

งาน	จาก	ไป	เหตุผล
t8 (แม่ 8)	cyolo2	cyolo1	cyolo2 ใช้ไป 32,595 โทเคน เกินงบ 1 จึงหยุดรับงานใหม่ งาน t4 ที่กำลังรันทำต่อจนเสร็จ

ผลงาน: ครบ 8/8 งาน

ตรวจไฟล์รวมด้วยสคริปต์แยก ครบ 96 บรรทัด (แม่ 2–9 คูณ 1–12) ไม่มีบรรทัดผิดหรือขาด แต่รูปแบบหัวข้อไม่เหมือนกันทุกแม่ เพราะไม่ได้สั่งรูปแบบไว้

สถิติแยกตามบัญชี

บัญชี	บทบาท	ครั้ง	สำเร็จ/ล้ม	input tokens	output tokens	เวลารวม (s)
yolo2	หัวหน้า (ล้ม), ลูกทีม	3	2/1	141,046	1,045	22.9
yolo1	หัวหน้า (หมดงบ), ลูกทีม	3	3/0	251,050	2,736	24.8
cyolo1	หัวหน้าที่ปิดงาน, ลูกทีม	4	4/0	380,350	1,485	72.7
cyolo2	ลูกทีม (หยุดรับงาน)	1	1/0	32,397	198	10.4

ครั้งที่ล้มของ yolo2 คือการล้มที่จำลองเอง

สรุป

ไม่มีสกิล vs มีสกิล

	ไม่มีสกิล	มีสกิล
วิธีเรียกแต่ละบัญชี	ลองผิดลองถูก เปิด --help	สคริปต์ใส่ flag ถูกและตรวจผลให้
บัญชีไหนใช้ได้	auth status (ไม่ตรงจริง)	check ด้วยการเรียกจริง
การทำงานร่วมกัน	สั่งทีละตัว ยัดข้อมูลลง prompt	relay · fan-out · graph · review · board · ขั้นสูง
สถิติ	ต้องเดา	ledger + โควตาพร้อมเวลาของข้อมูล
โควตาหมด	งานค้างหรือหาย	สลับหัวหน้าและย้ายงาน งานไม่หาย
เครื่องใหม่ / volume ใหม่	สร้างสกิลใหม่	คัดลอกโฟลเดอร์สำรองไปวาง (ขั้นที่ 2)
งานทีมชุดเดียวกัน	810.2 วินาที	170.5 วินาที

ประเด็นสำคัญ

จำ 3 ข้อนี้

1 · ความรู้

วิธีใช้ทุกบัญชีอยู่ใน **สกิล** ไม่ต้องลองผิดลองถูก

2 · สถานะ

งานและใบส่งเวรอยู่ใน **ไฟล์** โควตาหมดก็ไม่หาย

3 · ต้นฉบับ

โฟลเดอร์สำรอง **คัดลอกไปวาง** ได้ทุกเครื่อง

ข้อควรรู้

ข้อควรรู้

ล็อกอินเองเสมอ

สกิลไม่ล็อกอินแทน และไม่อ่าน .credentials.json / .claude.json / auth.json

ข้าม permission

ใช้กับงานที่อนุญาตเท่านั้น ตรวจผลของ agent อื่นซ้ำเสมอ

ไม่ retry อัตโนมัติ

งานที่ล้มหลังใช้โทเค็นแล้วหยุดและรายงาน

Git Bash แปลง path

ห่อด้วย `sh -c "…"` ตามตัวอย่างใน SKILL.md ใช้ได้ทั้ง Git Bash และ PowerShell

แก้ที่ต้นฉบับ

แก้ใน skills/ แล้วคัดลอกทับใน shared/skills อีกครั้ง

ติดตั้งครบ 3 โฟลเดอร์

team.py เรียก yolo.py และ cyolo.py ของสกิลข้างเคียงใน shared/skills เดียวกัน

คำสั่งที่ใช้บ่อย

ใน container

```
T=/root/ai-accounts/.Claude/shared/skills/agent-team/scripts/team.py
python3 $T check            # บัญชีไหนใช้ได้ (เรียกจริง)
python3 $T quota            # โควตาทุกบัญชี + เวลาของข้อมูล
python3 $T run graph.json   # รันกราฟงานจนจบ แล้วพิมพ์ตาราง + โควตา
python3 $T status           # ภาพรวมทีม
python3 $T stats            # สถิติตามบัญชี / รุ่น / run
```

คัดลอกสกิลเอง (จากโฟลเดอร์บท)

```
for d in .Claude .Codex; do for s in yolo cyolo agent-team; do
docker cp ./_solution/skills/$s multi-account:/root/ai-accounts/$d/shared/skills/; done; done
docker exec multi-account bash -lc 'for n in 1 2 3 4 5; do yolo$n --version; cyolo$n --version; done'
```

จาก host

```
docker exec multi-account sh -c "cat /root/ai-accounts/.Claude/shared/skills/agent-team/SKILL.md"
docker exec multi-account sh -c "python3 /root/ai-accounts/.Claude/shared/skills/agent-team/scripts/team.py stats"
docker exec multi-account sh -c "python3 /root/ai-accounts/.Claude/shared/skills/agent-team/scripts/team.py lead --demo --lead-model haiku"
```

เช็กลิสต์

ทำครบหรือยัง?

- [ ] มีโฟลเดอร์ `skills/` ที่มี yolo, cyolo, agent-team (ขั้นที่ 1)
- [ ] ทุกบัญชีเห็นสกิลครบ (ขั้นที่ 2)
- [ ] งานทีมชุดเดิมเร็วขึ้น และตัวเลขมาจาก ledger (ขั้นที่ 3)
- [ ] เห็นการสลับหัวหน้าทั้งเชิงรุก/เชิงรับ และการย้ายงานลูกทีม (ขั้นที่ 4)

สรุป

สอนครั้งเดียว ทุกบัญชีทำงานเป็นทีม

ความรู้อยู่ใน **สกิล** · สถานะอยู่ใน **ไฟล์** · ต้นฉบับอยู่ใน **โฟลเดอร์สำรอง**
หัวหน้าหรือลูกทีมโควตาหมด งานก็ไม่หาย

รายละเอียดทุกขั้นตอนและพรอมต์เต็มอยู่ใน README.md

